ADIA LAB · UNIVERSITY COLLEGE LONDON

Regime Detection using Sliced Wasserstein k-means Clustering with UnifOrtho

A thesis submitted for the partial fulfillment of the requirements for the degree in Master of Science in Computational Finance

Oscar Peyron

ADIA Lab · University College London

February 2026

Contents

Abstract

This thesis investigates the topic of regime detection in high dimensional time-series using an interpretable and efficient machine learning algorithm. In particular, we provide an enhanced method to cluster the multi-dimensional returns distributions building upon the Wasserstein clustering algorithm [8] and its high-dimensional generalization called the Sliced Wasserstein clustering algorithm proposed by [10]. One of the key characteristics of those clustering algorithms is their ability to infer which regime the market is currently in without pre-specified assumptions on the returns distribution. Specifically, they use the Wasserstein or Sliced-Wasserstein (in higher dimensions) distance as a metric to compare different returns distributions. Our focus will be on improving this Sliced Wasserstein clustering algorithm in high-dimensional settings. The latter method has been designed to project high-dimensional distributions into one-dimensional distributions and then compute the Wasserstein distance in one dimension, which is computationally efficient. However, in order to get a good approximation of the Sliced-Wasserstein distance, one needs to sample a large number of random projection vectors to approximate well the high-dimensional space. This problem is the central chore on which this thesis focuses. Instead of using a fixed set of well-chosen random projection vectors to compute the Sliced-Wasserstein distance as proposed by Luan et. al [10], we show that using the UnifOrtho method designed by Rowland et. al [14] to sample orthogonal projections leads to a reduction in variance (in general) as well as more stable clustering results in high-dimensions. Another, key contribution of this research is to use the clustering results from the improved Sliced-Wasserstein distance, to derive implied probabilities of regime swtiches in the market using simple softmax functions and bayesian priors. Finally, we design multiple trading strategies based on the above regime detection method and compare their performance.

1 Introduction

1.1 Background

Regime detection is a topic that studies the changes in the underlying dynamics of a financial time-series. Because financial time-series are notoriously non-stationary many static quantitative models fail to capture dynamics when faced to the real-world. This is why regime detection is such of great value for practitioners. On the buy side, funds and asset managers rely on diversification to hedge their clients against risk. However, this diversification relies on a stable correlation between assets, which is not the case during periods of crisis. One often observes that during extreme bear regimes cross-asset correlations converge towards 1. This is why regime detection is used as a risk management filter. For instance, switch periods from low/vol regimes to high/vol regimes are flagged to change portfolio allocation parameters. Sell side firms such as investment banks and brokers must handle large amounts of client orders and need to adapt their trading strategies. This is for instance, the case for market making execution logic which alters its VWAP/TWAP algorithms to adapt to micro-regimes. For instance, at an intraday level, the market can change from a mean-reverting regime to a trending regime and vice versa. Adapting to these macroscopic and microscopic regimes is crucial for practitioners to avoid losses and improve their performance.

1.2 Literature Review

Regime detection has become over the years a heavily studied topic in literature, as it is of great interest for practitioners. As explained above knowing regimes helps actively investors adapt their trading strategies and risk management to the current market conditions. One of the first notable papers by Hamilton [6] introduced a parametric approach to regime detection using hidden Markov models (HMM). This method is until now, one of the most widely used methods for regime detection in financial markets. Rydén et al. [15] showed that the HMMs approach captures heavy-tails and unconditional non-normality in the returns distriubtion, in addition to being a highly intuitive method for regime detection. More recently, Ang et al. [2] showed that regime in financial markets empirically switches abruptly and this changed behavior persists for many periods. However, HMMs struggle as the framework assumes that regimes are a mixture of Gaussian distributions, which is not always the case in financial markets. Moreover, breaking points in the markets forces the model to adapts its parameters for the return distribution, which can be difficult to do in practice. Traditional regime detection approach try to identify these global beaking-points for adapting parameters such as in Bai et al.[3] and with general-to-specific modeling approaches in Hendry et.al [7]. The Wasserstein -k-means approach as proposed by Horvath et al. [8] is instead classifying regime by analysing the behaviour of the *distribution* of the time-series of returns through the study period. This enables to be model-free for the returns distribution and therefore skip parameters adaptation issues. The Sliced Wasserstein k-means clustering algorithm as proposed by Luan et al. [10] generalizes Horvath's approach to high-dimensional time-series data and infers regimes changes using returns distribution in $\mathscr{P}_p(\mathbb{R}^d)$, the space of probability measures on $\mathbb{R}^d$ with finite p-th moment.

1.3 Motivations for this research

As practitioners are more and more interested in not only having models that work but also understanding them, interpretable models have become of great interest in the industry. One of the main contributions of the Sliced-Wasserstein k-means clustering algorithm as given by Luan et al. [10] and Horvath et al. [8] is to not only give a new method for regime detection, but also provides a key interpretability dimension to clustering regimes. This is because, the latter method fundamentally applies the well-known k-means clustering algorithm to high-dimensional distributions, making it highly interpretable. Moreover, with the emergence of portfolio trading – a trading strategy that trades a basket of up to 100 assets at the same time – it has become crucial to have regime detection methods that can handle high-dimensional time-series data. This additional component makes therefore not only interpretability crucial but also scalability. Thus, the applicability of the Sliced-Wasserstein k-means clustering algorithm in higher dimensions has become a key research topic in the field of regime detection.

2 Theoretical Background

This chapter will focus on the theoretical framework to understand the Wasserstein distance and its extension to the Sliced Wasserstein distance. It will also provide the mathematics behind the implied probability derived from the Wasserstein distance to produce predictive signals for trading strategies. Finally, we will provide some background on the statistical tests we used to compare the performance of the different trading strategies (i.e Ledoit-Wolf test on Sharpe ratios).

2.1 Theoretical Framework for Wasserstein Distance and Wasserstein Barycenters

To bether understand the Wasserstein distance and its extension to the Sliced Wasserstein distance, we will introduce a number of key concepts. In this thesis, we consider $\mathscr{X} = \mathbb{R}^d$ with d the dimension of the returns vector.

Definition: Data Streams A set $\mathscr{S}$ of streams of data over $\mathscr{X}$ is defined as:

$$S(\mathscr{X}) = \{(X_1, X_2, \ldots, X_N) | X_i \in \mathscr{X}, N \in \mathbb{N}\}.$$

For instance in this research we consider the following:

$$S = (S_1, S_2, \ldots, S_N) \in \mathscr{S}(\mathbb{R}^d) \tag{1}$$

where S_i might be the the price vector of the different assets at time i. [8]

Definition: Lifting Transformation A lifting transformation l is a map that maps the set of streams $\mathscr{S}(\mathscr{X})$ into the set of streams of streams:

$$l : \begin{cases} \mathscr{S}(\mathscr{X}) \to \mathscr{S}(\mathscr{S}(\mathscr{X})) \\ S \mapsto l(S) = (l_1, l_2, \ldots, l_M) \end{cases} \tag{2}$$

where $M > 1$. [8]

The example from [4] is the sliding window transformation:

$$l^m(\mathbf{x}) = (x_{1+h_2(m-1)}, x_{2+h_2(m-1)}, \ldots, x_{h_1+h_2(m-1)}) \text{ for } m = 1, 2, \ldots, M \tag{3}$$

with $M := \left\lfloor \frac{N-(h_1-h_2)}{h_2} \right\rfloor$ and $h_1, h_2 \in \mathbb{N}$ the window size and the step size respectively.

Definition: Discrete uniform distribution Given a stream generated by the lifting process $l(S) = (l^m(\mathbf{x})_i, : i = 1, 2, \ldots, h_1)$, we can define the discrete uniform distribution μ^S as follows:

$$\mu_m = \frac{1}{h_1} \sum_{i=1}^{h_1} \delta_{l^m(\mathbf{x})_i} \tag{4}$$

where we have that δ_x is the Dirac delta at point x. [10] This definition allows us to introduce the set of empirical distributions $\mathscr{K} = \{\mu_j\}_{1 \leq j \leq M}$ where $\mu_j \in \mathscr{P}_p(\mathbb{R}^d)$ with $\mathscr{P}_p(\mathbb{R}^d)$ the set of probability measures on $\mathbb{R}^d$ with finite p-th moment. [10].

Definition: Wasserstein distance $\mathscr{W}_p$ Let μ and ν be two probability measures on $\mathbb{R}^d$ with finite p-th moment defined as the following infimum over the joint distributions (X, Y) of d-dimensional random vectors X and Y. The Wasserstein distance of order p between μ and ν is defined as:

$$\mathscr{W}_p(\mu, \nu) = \inf_{(X,Y) \sim \Gamma(\mu,\nu)} (\mathbb{E}[\|X - Y\|^p])^{1/p} = \left(\inf_{\pi \in \Gamma(\mu,\nu)} \int_{\mathbb{R}^d \times \mathbb{R}^d} \|x - y\|^p d\pi(x, y) \right)^{1/p} \tag{5}$$

where $\Gamma(\mu,\nu) \subseteq \mathscr{P}_p(\mathbb{R}^d \times \mathbb{R}^d)$ is the set of joint distributions over $\mathbb{R}^d \times \mathbb{R}^d$ with marginals μ and ν. An element $\pi^* \in \Gamma(\mu,\nu)$ that achieves the infimum is called an optimal transport plan between μ and ν. [14]

Definition: Wasserstein barycentre $\bar{\mu}$ Given a family probability measures $\mathscr{K}$ in $\mathbb{R}^d$ with finite p-th moment, the Wasserstein barycentre $\bar{\mu}$ is defined as the probability measure that minimizes the sum of squared Wasserstein distances to the given measures:

$$\bar{\mu} = \arg \min_{\nu \in \mathscr{P}_p(\mathbb{R}^d)} \sum_{\mu_m \in \mathscr{K}} \mathscr{W}_p(\nu, \mu_m) \tag{6}$$

where $\mathscr{P}_p(\mathbb{R}^d)$ is the set of probability measures on $\mathbb{R}^d$ with finite p-th moment. [10]

Projected Empirical Measure Let $\theta \in \mathbb{S}^{d-1}$ with $\mathbb{S}^{d-1}$ the unit sphere in d dimensions. The projected empirical measure $\mu'(\theta) \in \mathscr{P}_p(\mathbb{R})$ is defined as:

$$\mu'(\theta) = \frac{1}{N} \sum_i \delta_{\langle x_i, \theta \rangle} \tag{7}$$

with x_i the atoms of the original empirical measure $\mu \in \mathscr{P}_p(\mathbb{R}^d)$ and $\langle \cdot, \cdot \rangle$ the standard inner product in $\mathbb{R}^d$. [10]

Definition: The sliced Wasserstein distance $\widetilde{\mathscr{W}_p}$ Given two distributions $\mu, \nu \in \mathscr{P}_p(\mathbb{R}^d)$ with $d > 1$, the sliced Wasserstein distance is defined as follows:

$$\widetilde{\mathscr{W}_p}(\mu,\nu) = \int_{\mathbb{S}^{d-1}} \mathscr{W}_p(\mu'(\theta), \nu'(\theta)) d\theta \tag{8}$$

with $\mathscr{W}_p(\mu'(\theta), \nu'(\theta))$ the Wasserstein distance in d=1 between the projected empirical measures $\mu'(\theta)$ and $\nu'(\theta)$ on the direction θ. [10]

In practice one can approximate the sliced Wasserstein distance using Monte Carlo methods by sampling L random directions $\{\theta^1, \dots, \theta^L\}$ uniformly on the unit sphere $\mathbb{S}^{d-1}$ and computing the average Wasserstein distance over these projections:

$$\widetilde{\mathscr{W}_p}(\mu,\nu) \approx \frac{1}{L} \sum_{l=1}^{L} \mathscr{W}_p(\mu'(\theta^l), \nu'(\theta^l)) \tag{9}$$

More precisely we have, $\theta^1, \dots, \theta^L \sim^{i.i.d.} \text{Unif}(\mathbb{S}^{d-1})$. [14] One of the thesis primary focus is to improve the sampling from $\text{Unif}(\mathbb{S}^{d-1})$ as it is not proprely done in Luan et al. [10].

Definition: The sliced Wasserstein Barycenter The sliced Wasserstein barycenter $\bar{\mu}$ of a family of probability measures $\mathscr{K} = \{\mu_j\}_{1\leq j\leq M}$ is defined as:

$$\bar{\mu}^{\widetilde{\mathscr{W}_p}} = \arg \min_{\nu\in\mathscr{P}_p(\mathbb{R}^d)} \sum_{\mu_m\in\mathscr{K}} \widetilde{\mathscr{W}_p^p}(\nu,\mu_m) \tag{10}$$

[10]

This barycenter will be used in the Sliced-Wasserstein k-means clustering algorithm for representing the centroids of the clusters.

2.2 Theoretical Framework for Sliced Wasserstein (SW) with UnifOrtho Projections

This section will provide insights on the mathematical background needed to understand the UnifOrtho approach for sampling projections vectors for the sliced Wasserstein distance. Notably, we will see that this method solves the low generalizable nature of using pre-defined projection vectors θ^l for the Sliced Wasserstein distance as proposed by Luan et al [10] and Horvath et al [8]. Instead the Unifortho approach provides a more robust and generalizable clustering algorithm in higher-dimensions.

As we have seen in the previous section, the Sliced Wasserstein distance is approximated using L sampling of random direction in the unit sphere $\mathbb{S}^{d-1}$ for a Monte Carlo estimation of the distance. This UnifOrtho method relies on the properties of the orthogonal group $O(d)$ and its associated Haar measure to sample those directions so that they are orthogonal two by two.

Definition: Orthogonal group $O(d)$ The orthogonal group $O(d)$ is defined as :

$$O(d) = \{U \in \mathbb{R}^{d\times d} \mid U^T U = I_d\} \tag{11}$$

Definition: Haar measure on $O(d)$ The Haar measure on $O(d)$ is the unique probability measure that satisfies the following properties:

1. $\mu(QA) = \mu(A)$ for all $Q \in O(d)$ and Borel sets $A \subseteq O(d)$ (left-invariance)
2. $\mu(AQ) = \mu(A)$ for all $Q \in O(d)$ and Borel sets $A \subseteq O(d)$ (right-invariance)
3. $\mu(O(d)) = 1$ (normalization)

Definition: UnifOrtho($\mathbb{S}^{d-1}$,N) Let $N \leq d$ be a positive integer. The probability distribution UnifOrtho($\mathbb{S}^{d-1}$,N) $\in \mathscr{P}((\mathbb{S}^{d-1})^N)$ is defined as the joint distribution of N rows of a random matrix drawn from the Haar distribution on the orthogonal group $O(d)$. In other words, if U is a random matrix drawn from the Haar distribution on $O(d)$, then the distribution of any N rows of U is UnifOrtho($\mathbb{S}^{d-1}, N$). [14]

Numerical Recipe for constructing matrices with Haar distribution on $O(d)$ from Mezzadri et. al[13]

1. Take an $N \times N$ complex matrix Z whose entries are complex standard normal random variables.
2. Feed Z into *any* QR decomposition routine. Let (Q,R), where $Z = QR$, be the output.
3. Create the following diagonal matrix

$$\Lambda = \begin{pmatrix} \frac{r_{11}}{|r_{11}|} & & \\ & \ddots & \\ & & \frac{r_{NN}}{|r_{NN}|} \end{pmatrix}, \tag{12}$$

   where the r_{jj}s are the diagonal elements of R.
4. The diagonal elements of $R' = \Lambda^{-1} R$ are *always* real and strictly positive, therefore the matrix $Q' = Q\Lambda$ is distributed with Haar measure.

Further explanation on understanding why the above recipe yields uniformly distributed orthogonal matrices in $O(d)$ can be found in [13].

Combining the above numerical schema [14, 13] as well as the general Sliced-Wasserstein k-means approach [10] we devise therefore the following pseudo-code.

Algorithm 1: sWk-means clustering with UnifOrtho Projections

Input: Data stream $S \in \mathscr{S}(\mathbb{R}^d)$,;
number of clusters K,;
number of projections L,;
tolerance ε for convergence
Output: K cluster assignments and corresponding 1d barycenters

▷ Preprocessing
Calculate $l(r^S)$ given $S \in \mathscr{S}(\mathbb{R}^d)$ and define family of empirical distributions $\mathscr{K} = \{\mu_j\}_{1\leq j\leq M}$;

▷ Step 1: UnifOrtho Projection Generation
$k \leftarrow \lceil L/d \rceil$; ▷ number of orthogonal bases needed
$\Theta \leftarrow \emptyset$;
for $i = 1$ **to** k **do**
  Sample a random matrix $Z_i \in \mathbb{R}^{d\times d}$ with i.i.d. $\mathscr{N}(0,1)$ entries;
  Compute the QR decomposition: $Z_i = Q_i R_i$;
  Let Λ_i be a diagonal matrix where $\Lambda_{i,jj} = \mathrm{sgn}(R_{i,jj})$;
  $U_i \leftarrow Q_i \Lambda_i$; ▷ U_i is Haar-distributed on $O(d)$
  Add the d column vectors of U_i to Θ;
$\{\theta^1,\ldots,\theta^L\} \leftarrow \Theta[1:L]$; ▷ select exactly L directions

▷ Step 2: Projection & K-Means Iteration
Calculate projected distributions $\{\mu_j'(\theta^l)\}_{1\leq j\leq M}$ for all $l = 1\ldots L$;
Initialize cluster assignments $C_1,\ldots,C_K$;
while *convergence criterion* $> \varepsilon$ **do**
  for $k = 1$ **to** K **do**
    Compute the 1d Wasserstein barycenter $\bar{\mu}_k(\theta^l)$ for each projection l using empirical distributions assigned to C_k;
  for $j = 1$ **to** M **do**
    Assign μ_j to the cluster C_{k^*} where $k^* = \arg\min_k \sum_{l=1}^{L} W_p^p\left(\mu_j'(\theta^l), \bar{\mu}_k(\theta^l)\right)$;
return $C_1,\ldots,C_K$ *and barycenters* $\{\bar{\mu}_k(\theta^l)\}$

One downside of this the Sliced-Wasserstein method is its high computational time complexity. Indeed, the UnifOrtho projection generation step alone is $\mathscr{O}(d^3)$ due to the QR decomposition of k matrices of size $d \times d$. In addition, the projection and k-means iteration step has a complexity of $\mathscr{O}(MKL)$ for the assignment step and $\mathscr{O}(KLM_k \log M_k)$ for computing the 1d Wasserstein barycenters. Here M_k refers to the number of distributions assigned to cluster C_k, K to the numbers of clusters and L to the number of projections.

One improvement that could be done is to use the Hadamard product, for reducing the time complexity to $\mathscr{O}(d^2 \log d)$ for generating the UnifOrtho projections by leveraging structured random matrices but this is beyond the scope of this work [5].

2.3 Variance of UnifOrtho Estimator of the Sliced Wasserstein Distance

As shown in Rowland et. Al (2019) [14], the variance of the Monte Carlo estimator of the sliced Wasserstein distance using UnifOrtho projections is not always a guarantee of variance reduction. In fact, Rowland et. Al (2019) [14] shows that there are cases in which the variance of the UnifOrtho are worse than an i.i.d Monte Carlo estimator. However, as shown in Petrovic et Al (2025) [16], the nature of the Wasserstein distance, and the fundamental properties of spherical harmonics, explain why the UnifOrtho estimator showcases lower variance for higher dimensions $d \geq 3$.

Proposition Let f be a continuous function on $\mathbb{S}^{d-1}$, and $(X_1|\ldots|X_d)$ be a matrix drawn from the Haar measure on the orthogonal group $O(d)$. Let Y_k^ℓ, $\ell \geq 0$, $1 \leq k \leq h_\ell$ be a basis of spherical harmonics, and $\hat{f}(\ell,k) = \int_{\mathbb{S}^{d-1}} f(x)\mathsf{Y}_k^\ell(x)\,\mathrm{d}x$ denote the spherical coefficients of f. Then

$$\operatorname{Var}\left(\frac{1}{d}\sum_{i=1}^{d} f(X_i)\right) = \frac{1}{d}\operatorname{Var}(f(X_1)) - \frac{d-1}{d}\sum_{\ell=1}^{+\infty}(-1)^{\ell-1}\lambda_{2\ell}\mu_{2\ell}(f) \tag{13}$$

$$= \frac{1}{d}\operatorname{Var}(f(X_1)) - \frac{d-1}{d}\sum_{\ell=1}^{+\infty}\lambda_{4\ell-2}\big(\mu_{4\ell-2}(f) - \alpha_{2\ell-1}\mu_{4\ell}(f)\big), \tag{14}$$

where

$$\mu_\ell(f) = \sum_{k=1}^{h_\ell}\hat{f}(\ell,k)^2, \quad \alpha_\ell = \frac{2\ell+1}{2\ell+d-1}, \quad \textit{and}\ \ \lambda_{2\ell} = \frac{\Gamma\left(\frac{d-1}{2}\right)\Gamma\left(\frac{2\ell+1}{2}\right)}{\sqrt{\pi}\,\Gamma\left(\frac{2\ell+d-1}{2}\right)}.$$

[16]

As explained in [16] we see that the variance of the UnifOortho estimator in 13 and 14 is reduced or increasing depending on the "energy profile" of $(\mu_{2\ell}(f))_{\ell\in\mathbb{N}}$ of the integrand f. One of the key insights shown by [16] is that for large values of d, we can use the generalize Sterling formula to approximate $\lambda_{2\ell} = \mathscr{O}(\ell^{-(d-1)/2})$. This yields that only the first terms in the alternating series in 13 and 14 is significant. This pulls the variance of the UnifOrtho estimator downwards and explains why the UnifOrtho estimator is more efficient in higher dimensions.

2.4 Theoretical Framework for Implied Probability Regime Detection Strategy

This thesis in addition to providing improvements on Luan et.al regime detection method [10] also investigates the use of the Sliced-Wasserstein distance to derive tradeable signals and predictions of future regime switches in the market. One major improvement this thesis proposes is the construction of likelihood function for determining the probability of being in a certain regime given the current empirical distribution of returns. In particular, we use SW-distance of an empirical distribution x_m to every centroid and then convert to a likelihood function via a softmax over the negative distances. We introduce a temperature parameter τ for controlling the sharpness of the likelihood function.

$$\mathscr{L}(\text{regime } k \mid x_m) = \frac{e^{-d(x_m,c_k)/\tau}}{\sum_j e^{-d(x_m,c_j)/\tau}} \tag{15}$$

with

$$x_m = 1/N \sum_{i=1}^{N} \delta_{l^m(\mathbf{x})_i}$$

the empirical distribution of the current window in $\mathscr{P}_p(\mathbb{R}^d)$ with a lifting transformation. After having run the Sliced-Wasserstein k-means clustering algorithm, on the target window of the data, we have for $k = 1, \dots, K$:

$$c_k^* = \arg \min_{\nu \in \mathscr{P}_p(\mathbb{R}^d)} \sum_{\mu_m \in C_k} \widetilde{W}_p^p(\nu, \mu_m), \quad k = 1, \dots, K \tag{16}$$

and we have for ν and μ in $\mathscr{P}_p(\mathbb{R}^d)$:

$$d(\nu, \mu) = \widetilde{\mathscr{W}_p}(\nu, \mu)$$

To construct a predictive signal we compute empirical transition matrix using a first-order Markov Chain with the historical labels of the regimes. Mathematically, one can define the probability of switching from regime i to regime j as follows:

$$\pi_{i,j} = P(Z_m = j \mid Z_{m-1} = i) = \frac{\text{Count}(i \to j)}{\sum_{x=1}^{K} \text{Count}(i \to x)} \tag{17}$$

If the market was $z_{current}$ our prior belief that state M is going to be at regime k is :

$$\mathbb{P}_{prior}(k) = \pi_{z_{current},k} \tag{18}$$

Then we simply use Bayes rule to derive the following :

$$\mathbb{P}_{Bayes}(k) = \frac{\mathbb{P}_{prior}(k) \cdot \mathscr{L}(k \mid X_m)}{\sum_{j=1}^{K} \mathbb{P}_{prior}(j) \cdot \mathscr{L}(j \mid X_m)} \tag{19}$$

We then apply simple probability principle to derive the probability of switching from the current regime to any other regime as follows:

$$\mathbb{P}_{switch} = 1 - \mathbb{P}_{Bayes}(z_{current}) \tag{20}$$

Another consideration when computing switching probability is to take into account the momentum of regimes. The underlying question that is asked is whether the market is drifiting away from the current regime or is it confirming the current regime. Mathematically, we translate this by considering a lookback window of size ω. One then computes the linear trend slope of the Wasserstein distance to each centroid k. Let $t \in \{1, 2, \ldots, \omega\}$ and using OLS, β_k we get:

$$\beta_k = \frac{\text{Covariance}(t, \mathscr{W}_2(X_{M-w+t}, C_k))}{\text{Variance}(t)} \tag{21}$$

- if $\beta_k < 0$ the distance to regime k is shrinking (we are getting close to regime k)
- if $\beta_k > 0$ the distance to regime k is growing (we are drifting away from regime k)

Again, we apply a softmax function to get a "stay" probability based on the magnitude and sign of β_k. We also introduce a new temperature parameter τ_{grad}.

$$P_{grad}(k) = \frac{\exp\left(-\frac{\beta_k}{\tau_{grad}}\right)}{\sum_{j=1}^{K} \exp\left(-\frac{\beta_j}{\tau_{grad}}\right)} \tag{22}$$

As the formula explains, the more negative the higher the $exp(-\beta_k)$ expression and the more likely we are to stay in regime k. Inversely, the more positive the β_k and the more likely we are to switch away from regime k.

The variant introducing momentum of regime is therefore a weighted average with the Bayes probability and expresses itself as follows:

$$\mathbb{P}_{switch} = 1 - [(1-\alpha)\mathbb{P}_{Bayes}(z_{current}) + \alpha \mathbb{P}_{Grad}(z_{current})] \tag{23}$$

with $\alpha \in [0:1]$ a weight that can be tuned on a validation set. The tuning of α is interesting because it implicitly indicates whether the predictive signal for regime switches is more driven by momentum or by the distance to the centroids. Tuning α for signal generation is therefore a key indication of the nature of the market and whether it is more driven by momentum or by mean-reversion.

2.5 Theoretical Framework for the Ledoit-Wolf Sharpe ratio test

As will we see in future sections, having stastistical tests to compare the performance of different trading strategies is of outmost importance. In the real world it allows practioners to understand whether the difference in performance between trading strategies is mere noise or whether it is a concrete improvement. In order to quantify this statistical significance, we will use the Ledoit-Wolf test on Sharpe ratios as proposed in [9]. We choose this test because it adapts to returns distributions that are heteroskedastic and autocorrelated, which is often the case in financial time series. We explain here the HAC inference framework form [9], which will be used later in the thesis.

Let us consider two trading strategies i and n with returns r_{ti} and r_{tn} for $t = 1, \ldots, T$. We therefore observe T return pairs (r_{ti}, r_{tn}) for $t = 1, \ldots, T$. Assuming that the the returns are stationary, we can define the following mean vector μ and covariance matrix Σ:

$$\mu = \begin{pmatrix} \mu_i \\ \mu_n \end{pmatrix}, \qquad \Sigma = \begin{pmatrix} \sigma_i^2 & \sigma_{in} \\ \sigma_{in} & \sigma_n^2 \end{pmatrix} \tag{24}$$

Let Sh_i and Sh_n denote the Sharpe ratio of the strategy i and n respectively. The quantity of interest is the difference in Sharpe ratios Δ and its estimator $\hat{\Delta}$:

$$\Delta \;=\; \text{Sh}_i - \text{Sh}_n \;=\; \frac{\mu_i}{\sigma_i} - \frac{\mu_n}{\sigma_n}, \qquad \hat{\Delta} \;=\; \frac{\hat{\mu}_i}{\hat{\sigma}_i} - \frac{\hat{\mu}_n}{\hat{\sigma}_n}. \tag{25}$$

In the framework proposed by Memmel (2003) [12] one works with the vector of means and covariances $u = (\mu_i, \mu_n, \sigma_i^2, \sigma_n^2)'$ and its estimator $\hat{u} = (\hat{\mu}_i, \hat{\mu}_n, \hat{\sigma}_i^2, \hat{\sigma}_n^2)'$. To derive the asymptotic distribution of $\hat{\Delta}$ using the delta method:

$$\sqrt{T} * (\hat{u} - u) \;\xrightarrow{d}\; N(0, \Omega) \tag{26}$$

With Ω the long-run covariance matrix of the vector u, which relies on the i.i.d return assumption.

Instead of working with the vector of means and covariances $u = (\mu_i, \mu_n, \sigma_i^2, \sigma_n^2)'$, we reparametrise the problem in terms of the vector of means and uncentred second moments as follows:

$$\gamma_i = \mathbb{E}(r_{1i}^2), \qquad \gamma_n = \mathbb{E}(r_{1n}^2), \qquad \upsilon = (\mu_i, \mu_n, \gamma_i, \gamma_n)', \qquad \hat{\upsilon} = (\hat{\mu}_i, \hat{\mu}_n, \hat{\gamma}_i, \hat{\gamma}_n)'. \tag{27}$$

And change the function f to express Δ and $\hat{\Delta}$ in terms of ν and $\hat{\nu}$:

$$\Delta = f(\upsilon), \qquad \hat{\Delta} = f(\hat{\upsilon}) \tag{LW 1}$$

with f defined as follows:

$$f(a,b,c,d) \;=\; \frac{a}{\sqrt{c-a^2}} \;-\; \frac{b}{\sqrt{d-b^2}} \tag{LW 2}$$

We assume in Ledoit et. al [9] that the following holds:

$$\sqrt{T}\,(\hat{\upsilon}-\upsilon) \;\xrightarrow{d}\; N(0,\Psi) \tag{LW 3}$$

where Ψ is a symmetric positive semi-definite matrix that is unknown and needs to be estimated.

From LW 3 and the delta method we can derive the asymptotic distribution of $\sqrt{T}\,(\hat{\Delta}-\Delta)$ as follows:

$$\sqrt{T}\,(\hat{\Delta}-\Delta) \;\xrightarrow{d}\; N\big(0,\, \nabla' f(\upsilon)\,\Psi\,\nabla f(\upsilon)\big) \tag{LW 4}$$

with the gradient of f that can be derived and which yields the following expression:

$$\nabla' f(a,b,c,d) \;=\; \left(\frac{c}{(c-a^2)^{1.5}},\; -\frac{d}{(d-b^2)^{1.5}},\; -\frac{1}{2}\frac{a}{(c-a^2)^{1.5}},\; \frac{1}{2}\frac{b}{(d-b^2)^{1.5}}\right) \tag{28}$$

Using the above, we obtain the standard error for $\hat{\Delta}$ is given by the following formula:

$$s(\hat{\Delta}) \;=\; \sqrt{\frac{\nabla' f(\hat{\upsilon})\,\hat{\Psi}\,\nabla f(\hat{\upsilon})}{T}} \tag{LW 5}$$

In Ledoit et. al [9] the authors condition this only on the existence of a consistent estimator $\hat{\Psi}$ of Ψ to derive the above error term.

The HAC inference framework estimates the limiting covariance matrix Ψ using the following equation:

$$\Psi \;=\; \lim_{T\to\infty} \frac{1}{T}\sum_{s=1}^{T}\sum_{t=1}^{T} \mathbb{E}\big[y_s y_t'\big]\,, \qquad y_t' = \big(r_{ti}-\mu_i,\; r_{tn}-\mu_n,\; r_{ti}^2-\gamma_i,\; r_{tn}^2-\gamma_n\big)\,. \tag{29}$$

Altenatively, the limiting covariance matrix Ψ can be expressed in terms of a function$\Gamma_T(j)$ as follows:

$$\Psi = \lim_{T\to\infty} \Psi_T, \qquad \Psi_T = \sum_{j=-T+1}^{T-1} \Gamma_T(j), \qquad \Gamma_T(j) = \begin{cases} \dfrac{1}{T}\displaystyle\sum_{t=j+1}^{T} \mathbb{E}\big[y_t y_{t-j}'\big] & \text{for } j \geq 0, \\[2ex] \dfrac{1}{T}\displaystyle\sum_{t=-j+1}^{T} \mathbb{E}\big[y_{t+j} y_t'\big] & \text{for } j < 0. \end{cases} \tag{30}$$

On then uses a real-valued kernel function $k(\cdot)$ as well as a bandwidth parameter S_T to compute a consistent estimator of the limiting covariance matrix Ψ. This method is called the HAC kernel estimator and is defined as follows:

$$\hat{\Psi} = \hat{\Psi}_T = \frac{T}{T-4} \sum_{j=-T+1}^{T-1} k\left(\frac{j}{S_T}\right) \hat{\Gamma}_T(j), \tag{31}$$

The correct kernel function must satisfy the following properties:

1. $k(0) = 1$,

2. k continuous at 0,

3. $\lim_{x \to \pm\infty} k(x) = 0$,

and with $\hat{\Gamma}_T(j)$ defined as follows:

$$\hat{\Gamma}_T(j) = \begin{cases} \frac{1}{T} \sum_{t=j+1}^{T} \hat{y}_t \hat{y}'_{t-j} & \text{for } j \geq 0, \\ \frac{1}{T} \sum_{t=-j+1}^{T} \hat{y}_{t+j} \hat{y}'_t & \text{for } j < 0, \end{cases} \qquad \hat{y}'_t = \left(r_{ti} - \hat{\mu}_i,\ r_{tn} - \hat{\mu}_n,\ r_{ti}^2 - \hat{\gamma}_i,\ r_{tn}^2 - \hat{\gamma}_n\right). \tag{32}$$

In particular, the estimator in 31 is robust to heteroskedasticity and autocorrelation in the returns, which is very useful when handling real financial time series returns data. We can then estimate the p-value for the null hypothesis $H_0 : \Delta = 0$ using the following formula:

$$\hat{p} \ = \ 2\Phi\left(-\frac{|\hat{\Delta}|}{s(\hat{\Delta})}\right) \tag{33}$$

with

$$\hat{\Delta} \pm z_{1-\alpha/2}\, s(\hat{\Delta}) \tag{34}$$

In the case of our study will be using the Quadratic-Spectral kernel as proposed in [1].

3 Experiment 1 - Synthetic Data Testing

The focus of this chapter will be on testing the algorithm on syntethic data to understand the behaviour of the algorithm and its performance in different settings. The algorithm will be tested on 3 dimensional data followed by 4, 5 dimensional. Performance between original and UniOrtho method will be compared and tests will span from data with multiple correlation factors and with diffrent target number of clusters. The synthetic data used in this experiment is the equivalent of 20 years of data with daily frequency. Therefore, it comprises $20 * 252 = 5040$ data points.

3.0.1 Performance Metrics

For this experiment, different performance metrics will be used to assess the quality of the regime clusters obtained by the Sliced-Wasserstein k-means clustering algorithm with UnifOrtho projections on the synthetic data.:

- Total Accuracy (TA)
- Mean centroid-centroid distance (MCCD)

.

The Total Accuracy (TA) as defined in [10] is defined as the proportion of correctly classified data points over the total number of data points. Mathematically, with each data returns points $r^S_{t_i} \in \mathscr{X}$ is assigned a label $\hat{y}_{t_i} \in \{1, \dots, K\}$, the Total Accuracy achieved by the clustering $\mathscr{C} = \{\hat{y}_{t_i}\}_{i=1}^N$ is defined as follows:

$$\mathrm{TA}(\mathscr{C}) = \frac{\sum_{i=1}^N \mathbb{I}_{\hat{y}_{t_i}=y_{t_i}}}{\sum_{i=1}^N (\mathbb{I}_{\hat{y}_{t_i}=y_{t_i}} + \mathbb{I}_{\hat{y}_{t_i}\neq y_{t_i}})} \tag{35}$$

where y_{t_i} is the true label of the data point $r^S_{t_i}$ and $\mathbb{I}$ is the indicator function.

Let the centroids of the clusters be denoted as $\{\bar{\mu}_k\}_{k=1}^K$, the Mean Centroid-Centroid Distance (MCCD) is defined as follows:

$$\langle \mathscr{W}_p(\bar{\mu}_k, \bar{\mu}_{k'}) \rangle_{k,k'} := (C_2^K)^{-1} \sum_{k'\neq k} \mathscr{W}_p(\bar{\mu}_k, \bar{\mu}_{k'}), \tag{26}$$

where $C_2^K = \binom{K}{2} = \frac{K(K-1)}{2}$ is the number of unique pairs of clusters. This metric represents how well-separated the different clusters are from each other in the $\mathscr{P}_p(\mathbb{R}^d)$ space. A higher MCCD indicates that the clusters are more distinct and less likely to overlap, which is desirable for effective clustering. Later for real datasets (i.e in an unsupervised setting) this metric will be used to evaluate the quality of the clustering as there will be no ground truth labels available for computing the Total Accuracy (TA). It is therefore important to consider how well the MCCD-metrics correlates with the Total Accuracy (TA) in the synthetic setting to understand whether it can be used as a proxy for evaluating clustering performance in real-world scenarios.

3.0.2 Experimental Setup and Results

Having considered the performance metric, we will generate synthetic data similarly to previous papers [8] and [10] using Geometric Brownian Motion (GBM) paths $S_t/S_{t-1} =$

$\exp(r_t^S)$ with $S_t = (S_t^1, \dots, S_t^d)$ and $r_t^S = (r_t^{S^{(1)}}, \dots, r_t^{S^{(d)}})$ where :

$$r_t^{S^{(i)}} \sim \mathcal{N}\left((\mu - \sigma^2/2)\mathbf{1}dt, \Sigma dt\right) \tag{36}$$

. with

$$\Theta = (\mu, \sigma^2) \tag{37}$$

And the covariance matrix $\boldsymbol{\Sigma}$ is defined as follows :

$$\Sigma_{ij} = \sigma^2\left(\delta_{ij} + (1 - \delta_{ij})\rho\right) \tag{38}$$

The δ_{ij} is the Kronecker delta, σ^2 is the variance of the returns and ρ is the correlation between the different assets.

Assuming, we want to cluster two different regimes (K=2), we define, as in previous papers [10], Bull and Bear regimes with the following parametrization :

- $\Theta_{\text{bull}} = (0.02, 0.2)$
- $\Theta_{\text{bear}} = (-0.02, 0.3)$

For K = 3, similarly to [10] we will consider the *moon-shape structure* generated using `datasets.make_moons` function from the `sklearn` library.

This is a summary similary to the one provided in [10] for the different market settings on which we are going to test the sWk-means clustering algorithm with UnifOrtho projections :

Table 1: Summary of synthetic dataset parameters with $d \in \{3, 4, 5, 100\}$

Type	Regime Bull	Regime Bear	Regime III (Bear/Moon)
Type A (K=2)	$\Theta_{\text{bull}}; \rho = +1/2$	$\Theta_{\text{bear}}; \rho = +1/2$	N/A
Type B (K=2)	$\Theta_{\text{bull}}; \rho = +1/2$	$\Theta_{\text{bull}}; \rho = -1/2$	N/A
Type C (K=3)	$\Theta_{\text{bull}}; \rho = +1/2$	$\Theta_{\text{bear}}; \rho = +1/2$	$\Theta_{\text{bear}}; \rho = -1/2$
Type D (K=3)	$\Theta_{\text{bull}}; \rho = +1/2$	$\Theta_{\text{bear}}; \rho = +1/2$	$\Theta_{\text{moon}}; \rho = +1/2$

As in Luan et al. [10] we have tested our Sliced-Wasserstein k-means clustering algorithm with UnifOrtho projections on the above mentioned synthetic data as well as with different tunings of the parameters of h_1, h_2 and L (resp. the window size, the step size and the number of projections). For purposes of brevity, we present in our thesis, only results for the case of $d = 5$. However, interested readers can find results for $d = 2, 3, 4, 100$ in the appendix. Each results in table is either the maximum total accuracy (TA), the median total accuracy and the total accuracy of the run with the maximum mean centroid-centroid distance (MCCD) over $N_S = 50$ runs of the Sliced-Wasserstein algorithm.

h_1	h_2	L	Max		Median		Max(MCCD)	
			A	B	A	B	A	B
20	4	9	0.9890	0.9985	0.9888	0.9985	0.9889	0.9985
		20	0.9896	0.9983	0.9895	0.9982	0.9895	0.9982
		40	0.9908	0.9986	0.9908	0.9986	0.9907	0.9986
		100	0.9889	0.9986	0.9888	0.9985	0.9888	0.9986
30	6	9	0.9950	0.9980	0.9950	0.9980	0.9949	0.9980
		20	0.9949	0.9984	0.9948	0.9983	0.9949	0.9983
		40	0.9951	0.9984	0.9948	0.9983	0.9948	0.9984
		100	0.9951	0.9982	0.9949	0.9982	0.9951	0.9982
40	8	9	0.9966	0.9981	0.9964	0.9980	0.9964	0.9979
		20	0.9965	0.9982	0.9964	0.9980	0.9963	0.9980
		40	0.9964	0.9982	0.9962	0.9980	0.9962	0.9980
		100	0.9968	0.9982	0.9967	0.9980	0.9966	0.9980
50	10	9	0.9965	0.9969	0.9962	0.9966	0.9965	0.9966
		20	0.9973	0.9974	0.9971	0.9973	0.9971	0.9972
		40	0.9973	0.9973	0.9971	0.9971	0.9971	0.9973
		100	0.9971	0.9973	0.9971	0.9973	0.9968	0.9973
60	12	9	0.9965	0.9963	0.9962	0.9962	0.9962	0.9960
		20	0.9960	0.9962	0.9957	0.9959	0.9960	0.9959
		40	0.9963	0.9965	0.9962	0.9964	0.9963	0.9965
		100	0.9960	0.9962	0.9957	0.9962	0.9957	0.9962

Table 2: Synthetic Type A/B (K=2), dimension d=5: Maximum total accuracy (MAX) and Maximum Mean-Centroid-Centroid Distance (MCCD) of the SW k-means clustering with UnifOrtho projections over 50 runs across window size h_1, step h_2 and number of projections $L \in \{9, 20, 40, 100\}$. Dark red ≈ 1, pale ≈ 0.5.

We observe that for data type A and B, the UnifOrtho projections yield very high accuracy and high MCCD for all tested hyperparameter settings. These results in Table 2 show that the UnifOrtho projections yield even better results than pre-defined projections in [10] with lower dimensions ($d = 2, 3$) for similar parametrization (h1, h2 and L). For instance, in Luan et.al [10] for data type A and B with $d = 2$ and parametrization $(h_1, h_2) \in \{(20, 4), (30, 6)\}$ only yields median total accurancy TA($\mathscr{C}$) ranging from 0.52 to 0.725 with $N_S = 100$ clustering runs. This contrast to the results in Table 2 with its stable numbers across parametrization above 0.99. Curious readers can also find that we observe similar results for $d = 2, 3, 4$ in the appendix in which for $h1 \geq 20$, total accuracy metrics and MCCD remain consistently high for type A and B synthetic data. This is a strong indication that the UnifOrtho projections not only is more efficient in higher dimensions but also generates more stable and accurate results across dimensions. Such phenomenon is also observed by noticing that the parameters h_1, h_2 and L have little impact on the accuracy of the model as opposed to the results in Luan et.al [10] in which the accuracy is highly sensitive to the choice of h_1, h_2 and L.

h_1	h_2	L	Max		Median		Max(MCCD)	
			C	D	C	D	C	D
20	4	9	0.9904	0.8633	0.5264	0.5264	0.9904	0.8633
		20	0.9915	0.8630	0.5208	0.5235	0.9915	0.8629
		40	0.9912	0.8670	0.5207	0.5220	0.9912	0.8670
		100	0.9936	0.8640	0.5244	0.5196	0.9934	0.8631
30	6	9	0.9964	0.8768	0.5073	0.5254	0.9964	0.8768
		20	0.9965	0.8767	0.5128	0.5307	0.9954	0.8767
		40	0.9968	0.8788	0.9967	0.5259	0.9968	0.8788
		100	0.9970	0.8789	0.5137	0.5374	0.9970	0.8788
40	8	9	0.9959	0.8793	0.5206	0.5269	0.9949	0.8793
		20	0.9967	0.8797	0.9951	0.5365	0.9959	0.8797
		40	0.9973	0.8794	0.7599	0.5230	0.9973	0.8792
		100	0.9973	0.8817	0.9953	0.5319	0.9956	0.8813
50	10	9	0.9943	0.8815	0.5387	0.5301	0.9930	0.8805
		20	0.9959	0.5311	0.7698	0.5241	0.9954	0.5302
		40	0.9965	0.8825	0.5503	0.5266	0.9942	0.8825
		100	0.9965	0.8799	0.5446	0.5299	0.9959	0.8799
60	12	9	0.9961	0.8814	0.7672	0.5289	0.9961	0.8814
		20	0.9959	0.8795	0.9938	0.5318	0.9956	0.8795
		40	0.9959	0.8807	0.7587	0.5357	0.9959	0.8805
		100	0.9963	0.5401	0.7630	0.5362	0.9963	0.5357

Table 3: Synthetic Type C/D (K=3), dimension d=5: Maximum total accuracy (MAX) and Maximum Mean-Centroid-Centroid Distance (MCCD) of the SW k-means clustering with UnifOrtho projections over 50 runs across window size h_1, step h_2 and number of projections $L \in \{9, 20, 40, 100\}$. Dark red ≈ 1, pale ≈ 0.5.

In Table 3, we observe that for data type C and D, the UnirOrtho struggles significantly more than for data type A and B. While maximum total accuracy remains very high for data type C, the median total accuracy is mostly around 50 % which indicates that the algorithm is not able to consistently classify the data points correctly. We observe in this context that the hyperparameter tuning of h_1, h_2 and L has a significant impact on the performance of the algorithm, with higher values of h_1 giving in general better results. While the Sliced-Wasserstein k-means clustering algorithm with UnifOrtho projections performs relatively well on GBM data even for $K = 3$, it struggles when faced with more complex patterns such as the moon-shape structure in type D. We observe that the median total accuracy for type D data is consistently around 50% across all tested hyperparameter settings,while proving relatively low Total Accuracy (TA) (86-88%), indicating that the algorithm is highly unstable for this type of data. A final interesting observation is that the MCCD metric is steadily very close to the maximum total accuracy metric for all types of synthetic data on which the Sliced-Wasserstein k-means algorithm is tested.

Below are plotted, the different regime detections plots for the different type of synthetcic data with $d = 5$, $h_1 = 30$, $h_2 = 6$ and $L = 20$. The plots show the regime detection for the Sliced-Wasserstein k-means clustering algorithm with UnifOrtho projections by taking the maximal of total accuracy accross $N_S = 50$ runs. In the caption of the plots, we provide the maximum total accuracy achieved by the Sliced-Wasserstein k-means algorithm as well as a balanced accuracy metric from `scikit-learn` to account for the imbalance in the number of data points corresponding to each regime. Red indicated periods represent the bear regime and green the bearish periods. The orange period indicate either the bear regime with negative correlation for type C or the moon-shape structure for type D as indicated in table 1.

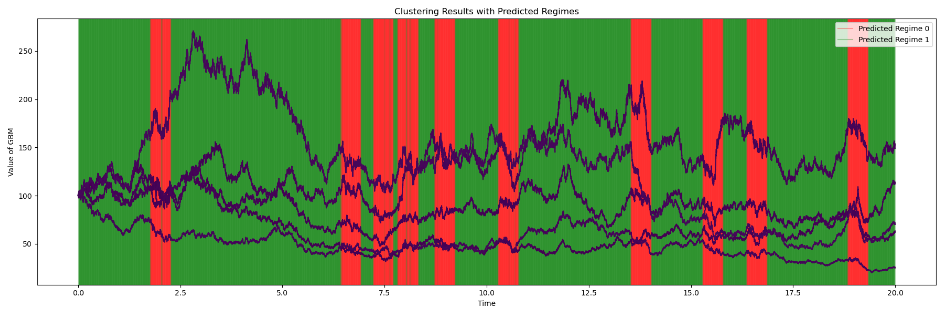


Figure 1: Regime detection plot for synthetic data type A with $d = 5$, $h_1 = 30$, $h_2 = 6$, and $L = 20$. The max total accuracy detected regimes by the Sliced-Wasserstein k-means clustering algorithm with UnifOrtho projections with $N_S = 50$. Total Accuracy(TA) = 0.9954, Balanced Accuracy(BA) = 0.9920, Mean Centroid-Centroid Distance(MCCD) = 0.0021.

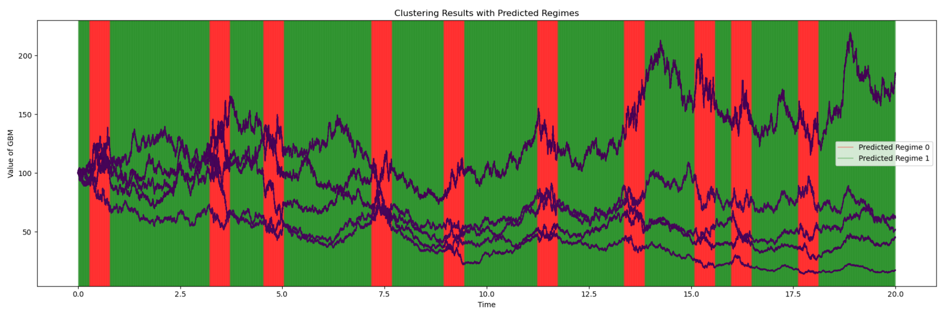


Figure 2: Regime detection plot for synthetic data type B with $d = 5$, $h_1 = 30$, $h_2 = 6$, and $L = 20$. The max total accuracy detected regimes by the Sliced-Wasserstein k-means clustering algorithm with UnifOrtho projections with $N_S = 50$. Total Accuracy(TA) = 0.9984, Balanced Accuracy(BA) = 0.9986, Mean Centroid-Centroid Distance(MCCD) = 0.0038.

We observe that the proposed regime detection method is able to detect the regimes in synthetic data type A and B with very high accuracy as well as high balanced accuracy. This is consistent with the results in Table 2 and shows that the Sliced-Wasserstein k-means clustering algorithm with UnifOrtho projections is able to detect regimes in synthetic data with high accuracy and stability across different hyperparameter settings.

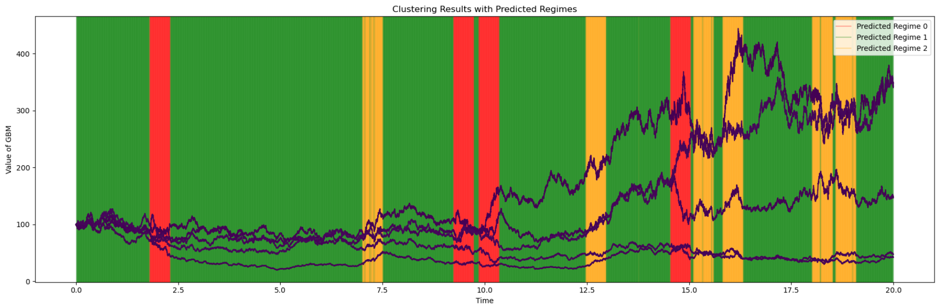


Figure 3: Regime detection plot for synthetic data type C with $d = 5$, $h_1 = 30$, $h_2 = 6$, and $L = 20$. The max total accuracy detected regimes by the Sliced-Wasserstein k-means clustering algorithm with UnifOrtho projections with $N_S = 50$. Total Accuracy(TA) = 0.9957, Balanced Accuracy(BA) = 0.9926, Mean Centroid-Centroid Distance(MCCD) = 0.0030.

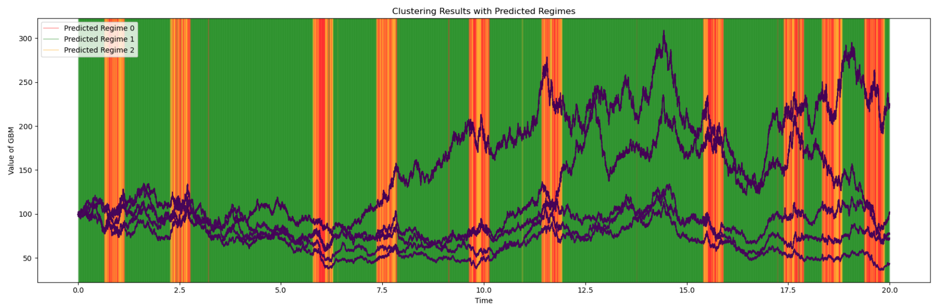


Figure 4: Regime detection plot for synthetic data type D with $d = 5$, $h_1 = 30$, $h_2 = 6$, and $L = 20$. The max total accuracy detected regimes by the Sliced-Wasserstein k-means clustering algorithm with UnifOrtho projections with $N_S = 50$. Total Accuracy(TA) = 0.8745, Balanced Accuracy(BA) = 0.6732, Mean Centroid-Centroid Distance(MCCD) = 0.0020.

While one observe particularly high accuracy and balanced accuracy for type C, we observe that type D struggles with only 87% total accuracy and 67% balanced accuracy. This is particularly noticeable when plotting the regime predictions for type D in Figure 4 where we observe that the regimes are visually not clearly separated in contrast to the other plots. The low balanced accuracy in the type D test also indicates that the SW-k-means clustering algorithm struggles to seperate the bear and moon regimes while it captures well the bull trend. Comparing type C and D, regime detections we can conclude that the Sliced-Wasserstein algo struggles with the moon shape structure and not necessarily with the number of regimes, as we observe that for three regimes in type C generated by GBMs the regime detection is still very accurate.

4 Experiment 2 - Application of sliced-SW k-means clustering with UnifOrtho projections on real world data sets

As synthethic data and particularly Geometric Brownian Motion do not accurately reflect real-world financial markets, this thesis also focuses on the application of the novel algorithm on real world data sets in financial markets and in particular on Equity Indices. Specifically, the focus will be on comparing and looking at the performance of the Sliced-Wasserstein k-means clustering algorithm with UnifOrtho projections across different regional clusters of Equity Indices. Those regional clusters include the following:

- **North America** S&P 500, NASDAQ, Dow Jones (US Equity Cluster) (daily data)
- **EMEA** CAC 40, DAX40, FTSE 100 (European Equity Cluster) (daily data)
- **Asia** Nikkei 225, Hang Seng, Kospi 200 (Asian Equity Cluster) (daily data)

For the sake of having statistically significant resutls, we consider 30 years of training data (1990- 2020) and 5 years of testing data (2020- 2025). This experiment will be split into three parts. First, we will look at an objective performance analysis of the regime detection on the real world data sets by using a long/short look-ahead trading strategy with signal generated using the current regime estimation. This will then alllow us to create real-world trading strategies with multiple variations for signal generation. The performance will then be compared to a long-only benchmark and a short-only benchmark. Finally, we will look at the performance of the different trading strategies under different hyperparameters to understand the impact of specific settings on the trading performance.

The parameters used for the developped trading strategy and regime detection for the first and second sub-experiment are given in the following table:

Symbol	Code variable	Value	Role
Sliced Wasserstein k-means (regime detection)			
h_1	`h1`	3	Lifting window length (support points per empirical distribution)
h_2	`h2`	1	Lifting stride between successive windows
L	`L`	20	Number of UnifOrtho projection directions
N_S	`N_S`	40	Number of random k-means restarts (best-MCCD run kept)
ε	`epsilon`	10^{-6}	Convergence tolerance on centroid movement
K	`K`	2^*	Number of regimes / clusters
General backtest & signal aggregation (all strategies)			
C_0	`initial_capital`	100	Initial capital
w	`_window_size`	20	Rolling window ($\approx$ 1 month) over which each regime is estimated
ℓ	`_majority_lookback`	20	Look-back length for the exponentially weighted majority vote
H	`_half_life`	5	Half-life of the exponential weights in the majority vote
Implied-probability signal (shared: hysteresis / continuous / conviction)			
τ	`_tau`	adaptive	Softmax temperature of the distance likelihood (`None` $\Rightarrow \frac{1}{2}$ mean inter-centroid distance)
g	`_use_gradient`	True	Whether to blend the regime-momentum (gradient) term
α	`_grad_weight`	$0.7^\dagger$	Weight of the gradient term in P_{switch}
ω	`_lookback`	20	Look-back for the OLS slope of distance-to-centroid (gradient)
τ_{grad}	`_tau_gradient`	adaptive	Softmax temperature of the gradient signal (`None` $\Rightarrow \frac{1}{2}$ std)
Hysteresis variant			
α_e	`_entry_threshold`	0.35	Switch-probability at/above which a counter-regime position is entered
α_h	`_hold_threshold`	0.21	Switch-probability below which the regime-aligned position is restored
Continuous variant			
δ	`dead_zone`	$0.10^\ddagger$	Dead-zone on $\pi_{\text{bull}} - \pi_{\text{bear}}$ below which the signal is set to 0
Conviction variant			
–	(slope)	$1.5^\ddagger$	Conviction decay $c = 1 - 1.5\,p_{\text{switch}}$

Table 4: Hyperparameters for the regime-detection and trading pipeline. * fixed by the two-regime setup, not a free parameter here. † `_grad_weight` is set but *inactive* while `_use_gradient=False`. ‡ hard-coded constant in the strategy code rather than a passed argument.

The above hyperparametrers are set as fixed for the look-ahead trading strategy, and all the other trading strategy implemented in this thesis.

In parallel to the traditional sharpe ratio, we will also be using the hit ratio defined below as a performance metric.

$$\text{Hit Ratio} = \frac{\text{Number of profitable trades}}{\text{Total number of trades}} \tag{39}$$

We use the hit ratio to evaluate the effectiveness of the trading strategies. A higher hit ratio indicates a higher proportion of profitable trades, which is desirable for a successful trading strategy. Another metric that will be considered is the win/loss ratio, which is defined as the ratio of the average profit of winning trades to the average loss of losing trades.

$$\text{Win/Loss Ratio} = \frac{\text{Average profit of winning trades}}{\text{Average loss of losing trades}} \tag{40}$$

The win/loss ratio provides insight into the risk-reward profile of the trading strategy. When this ratio exceeds 1, it indicates that the average profit from winning trades is greater than the average loss from losing trades. Finally, we need to introduce the metric filter in our study. This metric filter is used on real-world data sets to correctly assign labels 0 to bear regimes and 1s to bullish regimes. The issue arises from the fact the the Sliced-Wasserstein k-means clustering algorithm is only capable of separating regimes into different clustering labels, without providing insights on the nature of the distinct clusters. This is why in the thesis, we introduce a `choose_label` function that sorts the labels based on either CVaR, Skewness or the MeanVar ratio of the returns in each regime.

4.1 Performance Analysis of sliced Wasserstein Regime Detection using UnifOrtho with Look-Ahead

In this subsection we want to look at the performance of the regime detection algorithm using a trading strategy with look-ahead bias to understand whether the regime detection is able to detect the regimes on a day to day basis, without considering predicting signals.

Before looking at those consideration we can already visually observe in the plots below that the Sliced-Wasserstein regime detection algorithm labels well market periods for the three regions with downturns and upturns. Green represents below average market to bullish regimes, while red represents bearish regimes. The plots below were generated using the parameters in table 4 for the Sliced-Wasserstein regime detection algorithm.

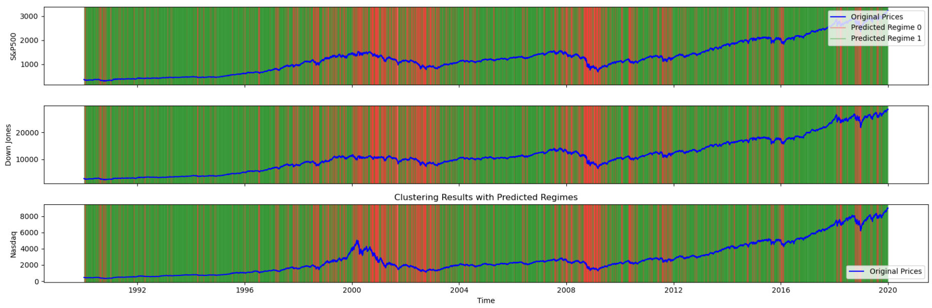


Figure 5: Regime labels ouptput by the Sliced-Wasserstein k-means clustering algorithm with UnifOrtho projections on the US equity indices (S&P 500, NASDAQ, Dow Jones) time series. Parameters $h_1 = 3$, $h_2 = 1$, $L = 20$, $N_S = 40$.

Figure 6: Regime labels ouptput by the Sliced-Wasserstein k-means clustering algorithm with UnifOrtho projections on the European equity indices (CAC 40, FTSE 100, DAX) time series. Parameters $h_1 = 3$, $h_2 = 1$, $L = 20$, $N_S = 40$.

Figure 7: Regime labels ouptput by the Sliced-Wasserstein k-means clustering algorithm with UnifOrtho projections on the Asian equity indices (Nikkei 225, Hang Seng, Kospi 200) time series. Parameters $h_1 = 3$, $h_2 = 1$, $L = 20$, $N_S = 40$.

The objective of this experiment is to transform the above qualitative observation into a quantitative benchmark of the regime detection performance on a real-world data set in which no "true-label" is available as market regimes are subject to interpretation. The pseudo code for the look-ahead trading strategy for this experiment is available in the appendix (2).

As stated above, we test our look-ahead trading strategy on the 3 regional inverse-vol weighted index portfolios. This is done again with an inital capital of 100, over the window 1990-2020 and using the above hyperparameters in 4.

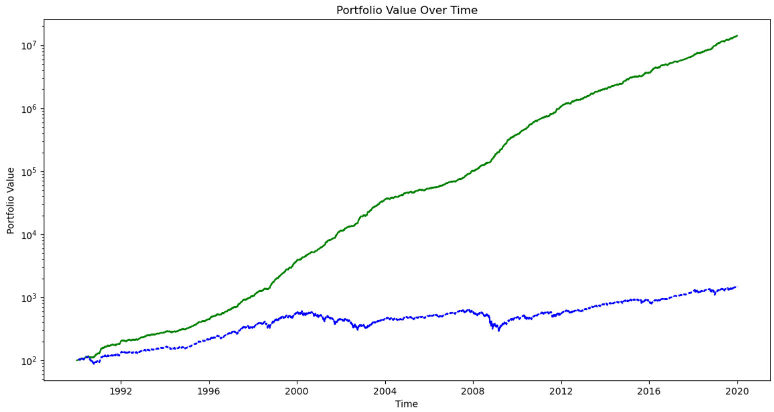


Figure 8: Look ahead trading strategy on the inverse vol weighted portfolio of US equity indices (S&P 500, NASDAQ, Dow Jones) with initial capital of 100. The trading strategy is based on the regime detection using the Sliced-Wasserstein k-means clustering algorithm with UnifOrtho projections. The trading strategy is a long/short strategy based on the current regime detection signal.

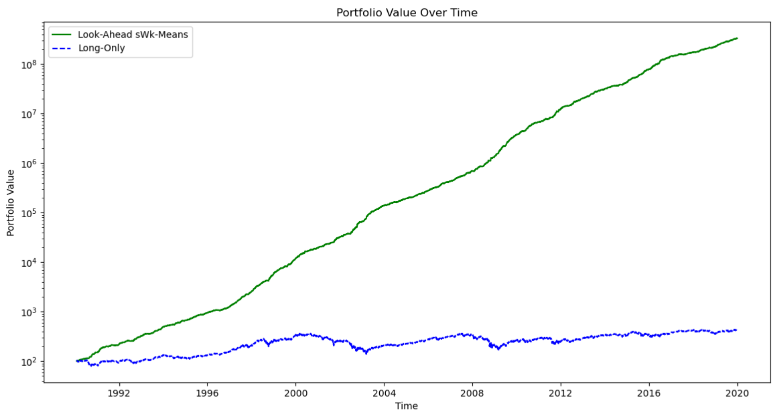


Figure 9: Look ahead trading strategy on the inverse vol weighted portfolio of European equity indices with initial capital of 100. The trading strategy is based on the regime detection using the Sliced-Wasserstein k-means clustering algorithm with UnifOrtho projections. The trading strategy is a long/short strategy based on the current regime detection signal.

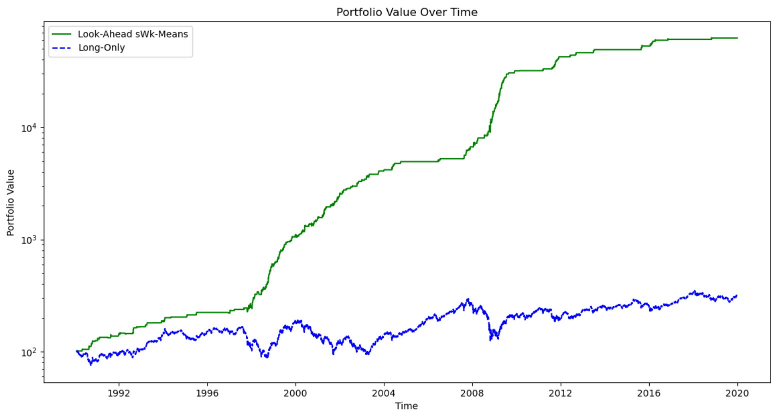


Figure 10: Look ahead trading strategy on the inverse vol weighted portfolio of Asian equity indices with initial capital of 100. The trading strategy is based on the regime detection using the Sliced-Wasserstein k-means clustering algorithm with UnifOrtho projections. The trading strategy is a long/short strategy based on the current regime detection signal.

Table 5: Cross-Regional Portfolio Performance Summary

Region	**Strategy Config**	**Metric Filter**	**Hit Ratio**	**Win/Loss**	**Ann. Sharpe**	**Ann. Return**
US Equity	Look-Ahead UnifOrtho	CVaR ($K = 2$)	0.5987	1.4994	3.9853	52.24%
EMEA	Look-Ahead UnifOrtho	CVaR ($K = 2$)	0.6066	1.6225	4.3411	64.92%
ASIA	Look-Ahead UnifOrtho	Skewness ($K = 2$)	0.6775	1.6299	1.99	23.95%

Note: Initial capital $I_0 = 100$.

- *US Equity* reached a final value of 29,911,844.77 (Cumulative PnL: 29,911,744.77).
- *EMEA* reached a final value of 329,792,991.92 (Cumulative PnL: 329,792,891.93).
- *ASIA* reached a final value of 62,503.61 (Cumulative PnL: 62,403.61).

We observe in the above three plots that the look-ahead trading algorithm generated from the labels of the Sliced-Wasserstein k-means clustering algorithm performs very well on the different equity indices. In particular, the Win/loss ratio is above 1.4 for all three regions, which indicates that the average profit from winning trades is significantly higher than the average loss from losing trades.

This give us a strong indication that under real empirical distributions the Sliced-Wasserstein k-means algorithm performs well. However, we observe that while performance is very good for the US and European markets it excells relatively less in Asian markets. This is particularly noticeable as US and European performance with a look-ahead trading strategy have an annualised Sharpe ratio of 4 or more, while the Asian look-ahead only reaches timidly 2.0 annualised Sharpe ratio. The performance gap can also be seen as the average annual return for the strategies in US and Europe go beyond 50% while the Asian strategy only reaches 23.95% average annual return. Qualitatively, we also observe on figures 8 and 9 that the Sliced-Wasserstein k-means clustering detects better bear regimes than the Asian counterparts, which translated in the performance gap in the look-ahead trading strategy. Interestingly, on the look-ahead trading strategy on the Asian equity indices we observe catastrophic results with we use the metric filter based on the CVaR metric. It seems that the CVaR captures badly bear regimes and misclassifies them as bullish regimes, which leads to a very poor performance of the trading strategy.

4.2 Performance analysis of regime detection using trading strategies

Having quantitatively and qualitatively established that the Sliced-Wasserstein k-means clustering algorithm performs well on real-world data sets, we now want to look at the performance of the Sliced-Wasserstein k-means algorithm for generating predictive signals for trading strategies. In this experience multiple trading strategies listed below are therefore considered.

- **Long/Short strategy** based on exponential weighted majority regime detection signals
- **Long/Short strategy** based on Implied probability signals (using the distance to the centroids as a proxy for the confidence of the regime detection)

  Hysteresis : Using threshold on the switch probability to enter long or short positions and to exit the position.

  Continuous : Using the raw probability signals to enter and exit positions without any thresholding.

  Conviction : Adjusting the position size based on the confidence of the regime detection.

- **Ensemble Methods**: combining the above four strategies using a weighted average of the signals or an adaptative weighting scheme to generate the final trading signal.

For curious readers all the above trading strategies are described in more detail in their respective pseudo codes in the appendix (3, 4). In our experimental settup we will be using the hyperparameters in table 4 for the regime detection and trading strategies. The trading strategies will be as above, tested on a portfolio of inverse-volatility weighted equity indices for the three regions (US, EMEA, Asia) and compared to a long-only benchmark as well as to a short-only benchmark. The time study will be from 1st of January 1990 to the 1st of January 2020 with daily data and an initial capital of 100.

We test two different ensemble strategies with different weighting schemes. One ensemble strategy with adaptative weighting based on the performance of the different trading strategies and one ensemble strategy with equal weights for all the trading strategies on the final signal.

Below, we plot the performance of the portfolio value applied to the different strategies for the three regions, as well as with the associated swtich probability as defined in 23 over time.

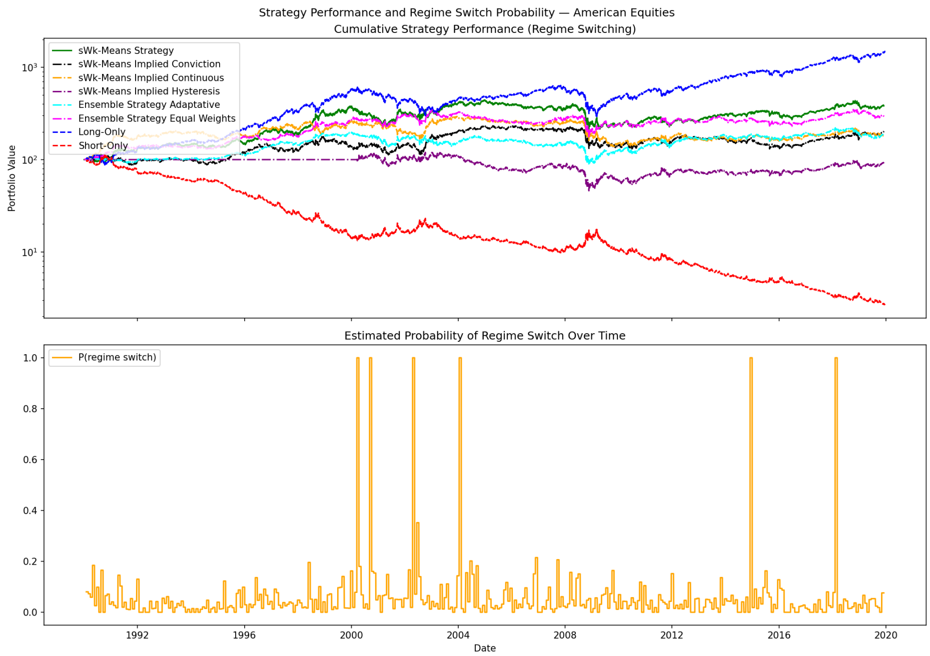


Figure 11: Look ahead trading strategy on the inverse vol weighted portfolio of American equity indices with initial capital of 100. The trading strategy is based on the regime detection using the Sliced-Wasserstein k-means clustering algorithm with UnifOrtho projections. The trading strategy is a long/short strategy based on the current regime detection signal.

Table 6: Compact summary: Sharpe ratio and annualised return of each strategy relative to the long-only benchmark, US equity cluster, 1990–2020.

Strategy	**Ann. Sharpe**	**Ann. Ret. (%)**	Hit ratio	Win Loss ratio
Long only (benchmark)	0.5992	9.41	–	–
Long/Short UnifOrtho	0.3427	4.58	0.5219	0.9765
Ensemble (equal)	0.3213	3.76	0.5272	0.9558
Implied Conviction	0.2205	2.30	0.5197	0.9627
Ensemble (Adaptative)	0.2137	2.08	0.5243	0.9462
Implied Continuous	0.2037	1.99	0.5149	0.9796
Implied Hysteresis	0.0571	−0.31	0.5142	0.9575
Short only	−0.5992	−11.39	–	–

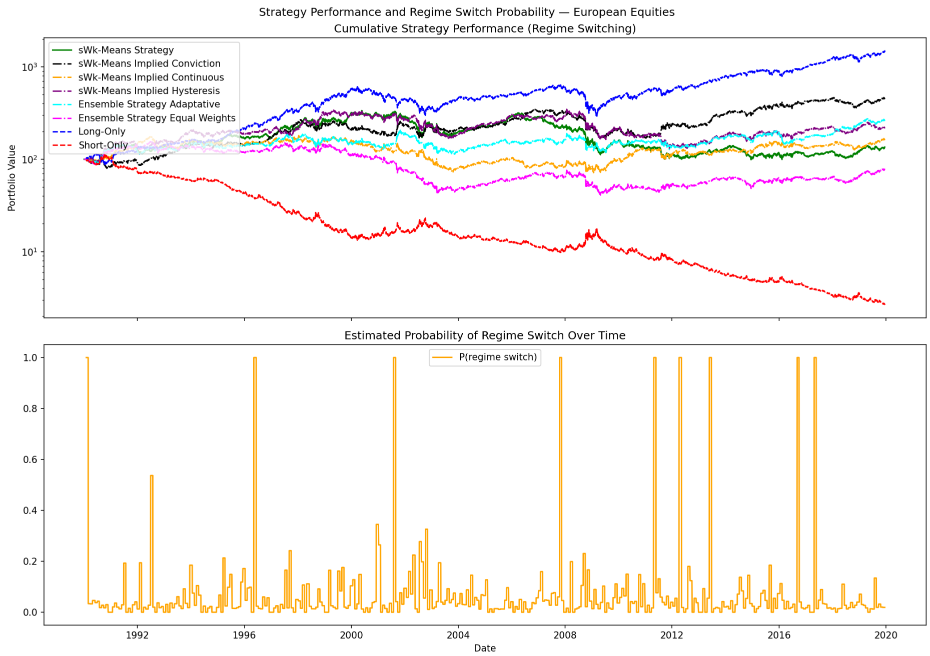


Figure 12: Look ahead trading strategy on the inverse vol weighted portfolio of European equity indices with initial capital of 100. The trading strategy is based on the regime detection using the Sliced-Wasserstein k-means clustering algorithm with UnifOrtho projections. The trading strategy is a long/short strategy based on the current regime detection signal.

Table 7: Compact summary: Sharpe ratio and annualised return of each strategy relative to the long-only benchmark, European equity cluster (CAC 40, DAX 40, FTSE 100), 1990–2020.

Strategy	**Ann.Sharpe**	**Ann. Ret. (%)**	Hit ratio	Win Loss ratio
Long only (benchmark)	0.5992	9.41	–	–
Implied Conviction	0.3774	5.12	0.5144	1.0107
Ensemble (Adaptative)	0.2787	3.23	0.5152	0.9907
Implied Hysteresis	0.2311	2.60	0.5149	0.9823
Implied Continuous	0.1784	1.59	0.5111	0.9872
Long/Short UnifOrtho	0.1474	1.02	0.5118	0.9796
Ensemble (equal)	0.0319	−0.78	0.5112	0.9620
Short only	−0.5992	−11.39	–	–

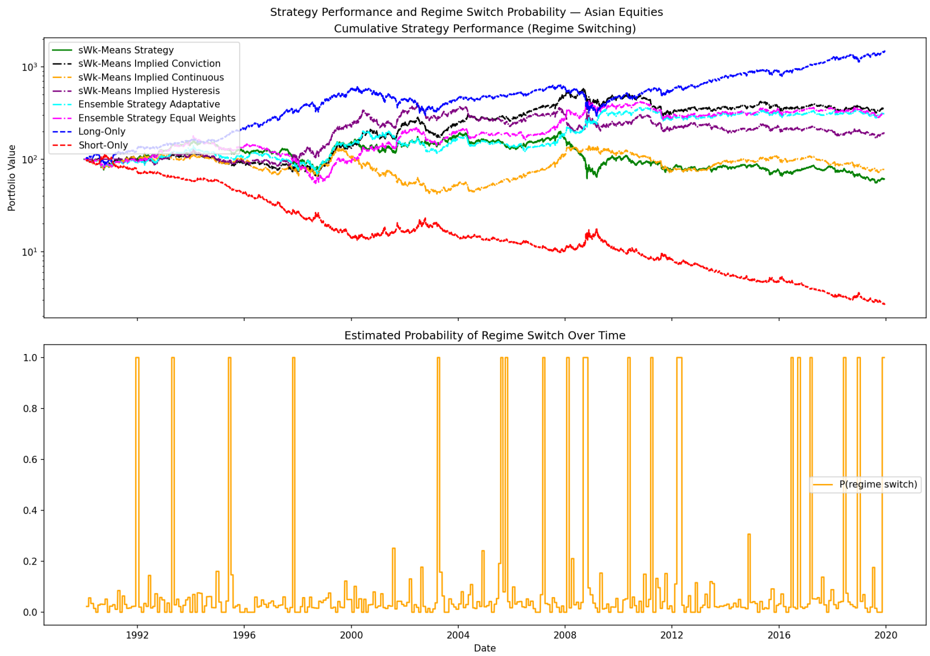


Figure 13: Look ahead trading strategy on the inverse vol weighted portfolio of Asian equity indices with initial capital of 100. The trading strategy is based on the regime detection using the Sliced-Wasserstein k-means clustering algorithm with UnifOrtho projections. The trading strategy is a long/short strategy based on the current regime detection signal.

Table 8: Compact summary: Sharpe ratio and annualised return of each strategy relative to the long-only benchmark, Asian equity cluster (Nikkei 225, Hang Seng, Kospi 200), 1990–2020.

Strategy	**Ann. Sharpe**	**Ann. Ret. (%)**	Hit ratio	Win Loss ratio
Long only (benchmark)	0.5992	9.41	–	–
Implied Conviction	0.3264	4.32	0.5122	1.0144
Ensemble (Adaptative)	0.3153	3.87	0.5078	1.0317
Ensemble (equal)	0.3131	3.80	0.5080	1.0282
Implied Hysteresis	0.2095	2.18	0.5069	1.0141
Implied Continuous	0.0275	−0.85	0.5042	0.9831
Long/Short UnifOrtho	0.0068	−1.63	0.5090	0.9660
Short only	−0.5992	−11.39	–	–

We observe on that for all the three regions, the Sliced-Wasserstein k-means clustering algorithm based trading strategies underperform the long-only benchmark. However, we need to note that those observations can only be made on the basis that the sharpe ratio difference is statistically significant. This is why in the next subsection we will conduct a Ledoit-Wolf test to compare the Sharpe ratios of the different trading strategies and see if the differences in performance are statistically significant.

4.2.1 Ledoit-Wolf test on sharpe ratios

In this thesis, we want to conduct a Ledoit-Wolf test to compare the Sharpe ratios of the different trading strategies with the long-only benchmark. The objective is to see if the differences in performance are statistically significant. To do so we conduct the Ledoit-Wolf sharpe ratio test as explained above with a confidence level of 95% and reported the p-values for each pairwise comparison between the trading strategies and the long-only benchmark. We use the HAC inference method with the Quadratic Spectral (QS) kernel as explained in section 2.5 to account for autocorrelation and heteroskedasticity in the returns. Under the above conditions and with the previously presented data we get the following results for the three regions (America, Europe, Asia):

Table 9: Ledoit–Wolf (2008) test of Sharpe ratio differences, US equity cluster (S&P 500, NASDAQ, Dow Jones). Benchmark: long-only, $SR_n = 0.0373$; $T = 7519$. HAC inference with prewhitened QS kernel.

Strategy	SR_i	ΔSR	t-stat	p-value	95% CI
Long/Short Unifortho	0.0216	−0.0157	−1.03	0.3045	[−0.0458, 0.0143]
Implied: conviction	0.0139	−0.0234	−1.51	0.1308	[−0.0538, 0.0070]
Implied: continuous	0.0128	−0.0245	−1.59	0.1118	[−0.0547, 0.0057]
Implied: hysteresis	0.0036	−0.0337	−2.21	0.0274*	[−0.0637, −0.0038]
Ensemble (adaptive)	0.0135	−0.0239	−1.56	0.1198	[−0.0539, 0.0062]
Ensemble (equal weights)	0.0203	−0.0171	−1.12	0.2638	[−0.0470, 0.0129]

* Rejects $H_0 : SR_i = SR_n$ at the 5% level.

Table 10: Ledoit–Wolf (2008) test of Sharpe ratio differences, European equity cluster (CAC 40, DAX 40, FTSE 100). Benchmark: long-only, $SR_n = 0.0369$; $T = 7399$.

Strategy	SR_i	ΔSR	t-stat	p-value	95% CI
Long/Short Unifortho	0.0094	−0.0275	−1.70	0.0887	[−0.0591, 0.0042]
Implied: conviction	0.0240	−0.0129	−0.80	0.4219	[−0.0442, 0.0185]
Implied: continuous	0.0113	−0.0255	−1.58	0.1134	[−0.0571, 0.0061]
Implied: hysteresis	0.0147	−0.0222	−1.38	0.1673	[−0.0536, 0.0093]
Ensemble (adaptive)	0.0177	−0.0191	−1.18	0.2374	[−0.0509, 0.0126]
Ensemble (equal weights)	0.0020	−0.0348	−2.17	0.0300*	[−0.0663, −0.0034]

* Rejects $H_0 : SR_i = SR_n$ at the 5% level.

Table 11: Ledoit–Wolf (2008) test of Sharpe ratio differences, Asian equity cluster (Nikkei 225, Hang Seng, KOSPI 200). Benchmark: long-only, $SR_n = 0.0346$; $T = 6739$.

Strategy	SR_i	ΔSR	t-stat	p-value	95% CI
Long/Short Unifortho	0.0005	−0.0342	−2.01	0.0448*	[−0.0676, −0.0008]
Implied: conviction	0.0217	−0.0129	−0.76	0.4457	[−0.0460, 0.0203]
Implied: continuous	0.0018	−0.0328	−1.91	0.0559	[−0.0665, 0.0008]
Implied: hysteresis	0.0140	−0.0207	−1.22	0.2216	[−0.0539, 0.0125]
Ensemble (adaptive)	0.0210	−0.0136	−0.80	0.4262	[−0.0472, 0.0200]
Ensemble (equal weights)	0.0209	−0.0138	−0.80	0.4209	[−0.0473, 0.0198]

* Rejects $H_0 : SR_i = SR_n$ at the 5% level.

4.3 Tuning the hyperparameters for the trading strategies

Tested multiple values for the hyperparameters of the trading strategies, such as the number of projection vectors L, the entry and hold thresholds, the half-life parameter for the exponentially weighted majority voting scheme, the weights for the ensemble methods and the gradient weight for the implied probability strategy with gradient.

4.3.1 Tuning the half-life parameter for the exponentially weighted majority voting scheme

Tested multiple values for the half-life parameter τ in the exponentially weighted majority voting scheme for

- the standard long/short unifortho strategy and
- the implied probability strategy with hysteresis, continuous and conviction variations.
- the ensemble method with equally weighted signals.

The values tested are the following

$$\tau \in \{1.0, 3.0, 5.0, 10.0, 20.0\} \tag{41}$$

The tau value is selected based on its hit ratio and win/loss ratio during the backtesting period (2006-2020). Results:

- Long/Short standard : $\tau = 5.0$
- Implied Strategies : $\tau = 5.0$
- Ensemble method : $\tau = 5.0$

4.3.2 Tuning the weights for ensemble methods

Tested multiple combination of weights for the ensemble methods, which combines the binary regime long/short stratey and the implied probability strategy with different variations (hysteresis, continuous, conviction). The weights are ordered as following :
$[w_{binary}, w_{implied\_probability(hysteresis)}\ w_{implied\_probability(continuous)}, w_{implied\_probability(conviction)}]$. We have tested the following weights:

- [0.25, 0.25, 0.25, 0.25],
- [0.30, 0.10, 0.30, 0.30],
- [0.20, 0.30, 0.20, 0.30],
- [0.30, 0.20, 0.30, 0.20],
- [0.10, 0.40, 0.10, 0.40]

4.3.3 Tuning the gradient weight for implied probability methods

Tested multiple values for the gradient weight α in the implied probability strategy with gradient. The values tested are the following

$$\alpha \in \{0.0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1.0\} \tag{42}$$

Optimal value for α is determined based on the highest Sharpe ratio achieved during the backtesting period. For the American equity data we have determined that the optimal value is $\alpha = 0.7$. As most of the variants of implied porbability strategies (continus and hysteresis), have a peak performance at $\alpha = 0.7$.

$\alpha = 0.0$ corresponds to a pure bayesian approach. It captures current cluster membership, regime persistence and more generally where the market is. $\alpha = 1.0$ corresponds to a pure gradient approach. It captures the direction of the market (getting closer or further from a certain regime) and therefore captures the momentum of the market.

Hence an optimal value of $\alpha = 0.7$ suggests that regime momentum is slightly more important than the current regime membership and regime persistence for predicting the next day return of the market. We could also thinkg that $\mathbb{P}_{Bayers}$ and $\mathbb{P}_{Grad}$ are not redundant and capture different information about the market.

4.3.4 Tuning the temperature parameter τ and τ_{grad} for the implied probability strategy

Peformed grid search for τ taken as adaptative to the switch probability as well as for τ_{grad} taken as adaptative to the gradient values. The optimal values for τ and τ_{grad} are determined based on the highest hit ratio achieved during the backtesting period (2006-2020). For the American equity data, we have determined that the optimal value for the couple (τ, τ_{grad}) must be around $(0.5, 0.1)$ overall based on all the trading strategies.

5 Experiment 3: Interpretability Analysis of Sliced Wasserstein Regime Detection Algorithm

While it is of great interest to practioners to generate signal from regime detection algorithms, is it almost equally important in the industry to understand the output of the Sliced-Wasserstein k-means clustering method. In particular, having an interpretation of the outputs in terms of macroeconomic indicators can be of great interest to understand the applicability of such regime detection method. We want to specifically understand whether the statstical and mathematical methods used to detect bear regimes coincide with real periods of economic downturns and market uncertainty.

5.1 SHAP Analysis

To do so, we analyse the feature importance of different macro-indicators during predicted bear regimes on the output of supervised learning modelsusing the SHAP analysis toolbox introduced by [11]. In order to understand the SHAP methodology, we introduce the following notations. Let us denote by f the supervised learning model, and let $F = \{1, \ldots, M\}$ be the set of features – in our case the macroeconomic indicators – and let $x = (x_1, \ldots, x_M) \in \mathbb{R}^M$ be a specific input. For any subset of features $S \subseteq F$ we denote by x_S the values of the features in S and by $x_{\bar{S}}$ the values of the features not in S. We define the value function $v(S)$ as :

$$v(S) \;=\; \mathbb{E}\big[\, f(X) \mid X_S = x_S \,\big], \qquad S \subseteq F = \{1, \ldots, M\}. \tag{43}$$

[11]

The Shap value or Shapley value ϕ_i of the i-th feature is then defined as the average marginal contribution of the i-th feature to all possible subsets of features $S \subseteq F \setminus \{i\}$, weighted by the size of the subset S and the total number of features M:

$$\phi_i \;=\; \sum_{S \subseteq F \setminus \{i\}} \frac{|S|!\,\big(M - |S| - 1\big)!}{M!} \Big[\, v\big(S \cup \{i\}\big) - v(S) \,\Big]. \tag{44}$$

In the following experiment we use the SHAP value because of its properties as described in [11]. Indeed, the Shapley value is the only feature attribution method which satisfies:

- **Local accuracy**: The shap value decomposes the individual prediction exactly:

$$f(x) = \phi_0 + \sum_{i=1}^{M} \phi_i \tag{45}$$

- **Missingness**: A feature absent from on instance receives no attribution $\phi_i = 0$.
- **Consistency**: If a model changes so that the marginal contribution of a feature increases or stays the same regardless of other features, the attribution for that feature should not decrease.

Those properties intuitively describe what a feature contribution should satisfy and therefore make the shap value a perfect candidate for our analysis.

5.2 Experimental Setup and Results

In our experiment we use the regime detection algorithm on weekly equity data (S&P 500, NASDAQ, Dow Jones) on the period from the 6th of Jan 2006 to 20th of December 2019. The Sliced-Wasserstein regime detection algorithm is then applied with the parameters $h_1 = 7, h_2 = 3, L = 20, N_S = 100, \varepsilon = 10^{-6}, K = 2$. To help us in the interpretation of the regimes we consider the following list of macroeconomic indicators on the study period.

- VIX Index
- 10-year Treasury Yield
- 5-year Treasury Yield
- 30-year Treasury Yield
- Fed Funds Rate
- Manufacturing PMI
- Manufacturing Employment
- DXY Dollar Index
- Gold Spot Price
- Brent Crude Oil Spot Price

This list is used as feature inputs for the supervised learning methods below to fit onto the regime classification generated by the Sliced-Wasserstein k-means clustering algorithm.

- OLS Linear Regression (probabilistic target (Probability of being in bear regime))
- Logistic Regression (binary target (Bull/Bear))
- Random Forest Classifier (binary target (Bull/Bear))

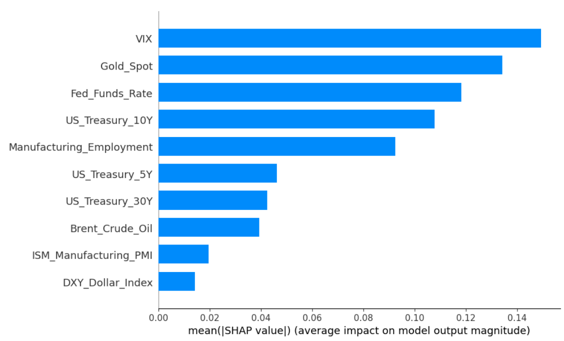
In the OLS Linear Regression, we use the probability of being in a bear regime as the target variable. This probability is computed using the regime labels generated by the Sliced-Wasserstein k-means method. Those labels are then transformed into a likelihood of being in a bear regime by using the softmax funtion introduced in equation 15. From this probabilistic target, we can then fit the OLS linear regression model to understand the relationship between the macroeconomic indicators and the probability of being in a bear regime. In the two other supervised learning methods (Logistic Regression and Random Forest Classifier), we use the binary regime labels generated by the Sliced-Wasserstein k-means method directly as the target variable. We encode the bear regime as 1 and the bull regime as 0.

We observe the following accuracy score for the three supervised learning methdos on the training data :

- OLS Linear Regression ROC(AUC) score: 0.980
- Logistic Regression Balanced Accuracy score: 0.702
- Random Forest Classifier Out-of-Bag balanced accuracy: 0.867

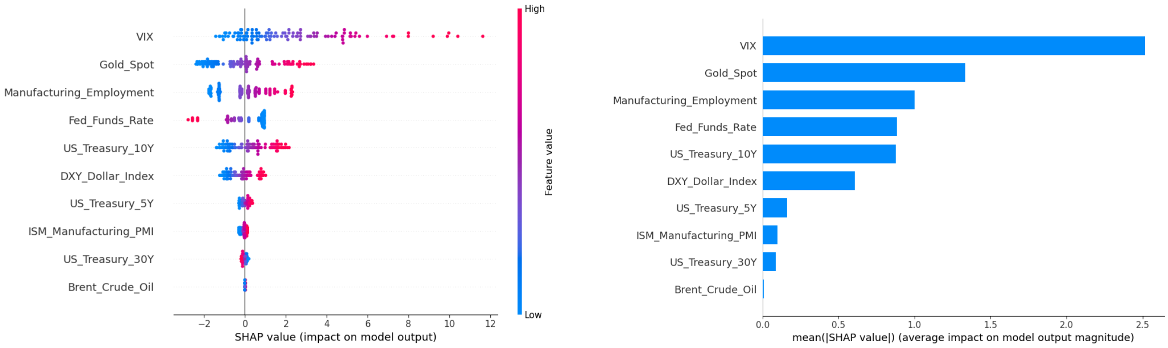
Below we present the features importance ranking as well as SHAP values of the features for the OLS Linear Regression model, Logistic Regression, and Random Forest Classifier. Those plots below are a feature restriction on bear labelled market periods. In other words, out of the 729 rows that contains the weekly data from 2006 to 2019, we only consider bear rows (i.e. rows where the regime label is 1 or in which the probability of being bear is greater than 0.5) for our analysis which are 110 rows.

(a) SHAP analysis summary.



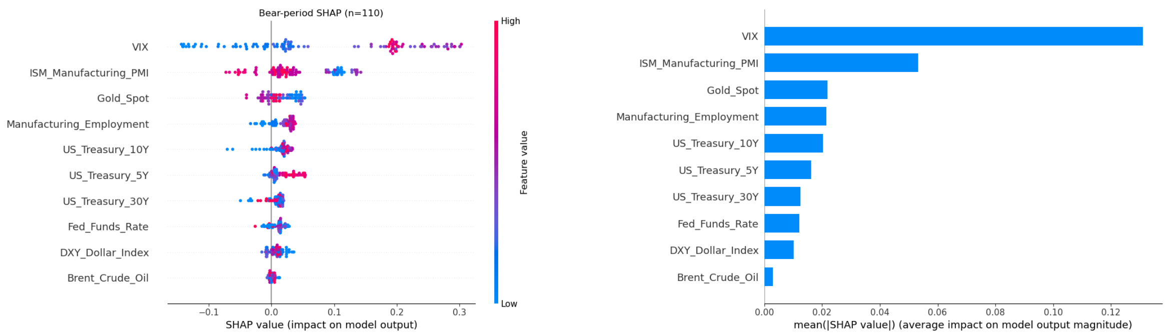
(b) Feature importance ranking.

Figure 14: Feature importance analysis for the OLS Linear Regression model fitted on the probabilistic target (Probability of being in bear regime). The features are ranked based on their absolute coefficient values.



(a) SHAP analysis summary. (b) Feature importance ranking.

Figure 15: Feature importance analysis for the Logistic Regression model fitted on the probabilistic target (Probability of being in bear regime). The features are ranked based on their absolute coefficient values.



(a) SHAP analysis summary. (b) Feature importance ranking.

Figure 16: Feature importance analysis for the Random Forest model fitted on the probabilistic target (Probability of being in bear regime). The features are ranked based on their absolute coefficient values.

We observe that for the method during bear periods the most important macroeconomic indicators is the VIX index. This is consistent with the literature on financial markets, as the high VIX sensitivity indicates high market volatility and uncertainty which is typically associated with bear markets. The Linear Regression and Logistic Regression models also show that the Gold Spot is of high importance during bear periods. The high Gold-Spot sensitivity also empirically coincides with bear markets as investors tend to rush to safe-haven assets such as gold during periods of market stress. Not only the feature rankings but also the SHAP analysis summary plots show this trend. We observe in the latter that elevated VIX and Gold values (the red dots in the plots) push the model's output towards a higher probability of being in a bear regime (or towards a bear classification). Those high values also coincide with high shape value which shows a strong positive contribution of those features to the model's output.

On the other hand, we observe that the Fed Funds Rate captures high negative shap values when it is high for Linear Regression and Logistic Regression models. This suggests that higher Fed Funds Rate values push the model's output towards lower probability of being in a bear regime (or towards a 0 bull classification) which is also consistent with the literature as higher interest rates are typically associated with a strong economy and low market volatility.

6 Conclusion

As we have seen in this thesis, the Sliced-Wasserstein k-means clustering algorithm is a well-suited method for detecting regimes in finacial markets whether on synthetic or real-world data sets. Adding, sampling projections vectors using UnifOrtho elevates total accuracy in high-dimensional settings ($d \geq 3$) and also for low-dimensional settings ($0 < d \leq 2$) compared to performance of the Standard Sliced-Wasserstein k-means clustering algorithm with pre-specified projections (i.e Luan et al. 2021 [10]). Even better, when applying the Sliced-Wasserstein k-means clustering algorithm on look-ahead trading strategies, we observe expecptional performance indicating that the bear and bull regimes are well distingushed by the method.

A Appendix

A.1 Sliced Wasserstein on Synthetic Data - Performance Tables

h_1	h_2	L	Max		Median		Max(MCCD)	
			A	B	A	B	A	B
10	2	2	0.5109	0.9715	0.5108	0.9715	0.5098	0.9711
		4	0.5858	0.9763	0.5141	0.9763	0.5858	0.9762
		9	0.5080	0.9442	0.5079	0.7253	0.5080	0.9442
		16	0.5091	0.9672	0.5087	0.7471	0.5090	0.9672
20	4	2	0.9744	0.9677	0.9741	0.9677	0.9740	0.9677
		4	0.9650	0.9947	0.9649	0.9946	0.9648	0.9946
		9	0.9734	0.9934	0.9733	0.9933	0.9733	0.9934
		16	0.9705	0.9954	0.9703	0.9953	0.9703	0.9954
30	6	2	0.9884	0.9862	0.9884	0.9860	0.9881	0.9853
		4	0.9864	0.9969	0.9863	0.9968	0.9863	0.9967
		9	0.9836	0.9939	0.9834	0.9939	0.9836	0.9939
		16	0.9858	0.9967	0.9858	0.9965	0.9858	0.9965
35	7	2	0.9895	0.9877	0.9895	0.9876	0.9895	0.9870
		4	0.9917	0.9962	0.9916	0.9961	0.9917	0.9962
		9	0.9883	0.9952	0.9881	0.9952	0.9883	0.9952
		16	0.9907	0.9957	0.9897	0.9955	0.9907	0.9957

Table 12: Synthetic Type A/B (K=2), dimension d=2: total accuracy of the SW k-means clustering with UnifOrtho projections across window size h_1, step h_2 and number of projections $L \in \{2, 4, 9, 16\}$. Dark red ≈ 1, pale ≈ 0.5.

h_1	h_2	L	Max		Median		Max(MCCD)	
			A	B	A	B	A	B
20	4	9	0.9857	0.9987	0.9856	0.9987	0.9856	0.9986
		16	0.9868	0.9980	0.9868	0.9980	0.9868	0.9980
		20	0.9811	0.9986	0.9810	0.9985	0.9810	0.9985
		40	0.9818	0.9979	0.9816	0.9978	0.9817	0.9978
30	6	9	0.9923	0.9981	0.9923	0.9981	0.9922	0.9979
		16	0.9927	0.9986	0.9926	0.9984	0.9925	0.9984
		20	0.9923	0.9978	0.9922	0.9978	0.9922	0.9977
		40	0.9935	0.9983	0.9934	0.9982	0.9935	0.9982
40	8	9	0.9965	0.9973	0.9964	0.9971	0.9965	0.9973
		16	0.9969	0.9980	0.9967	0.9980	0.9967	0.9980
		20	0.9960	0.9981	0.9959	0.9981	0.9960	0.9979
		40	0.9962	0.9981	0.9960	0.9981	0.9960	0.9981
50	10	9	0.9962	0.9967	0.9959	0.9967	0.9959	0.9964
		16	0.9964	0.9977	0.9964	0.9977	0.9964	0.9977
		20	0.9964	0.9973	0.9962	0.9973	0.9961	0.9973
		40	0.9964	0.9972	0.9963	0.9969	0.9964	0.9972
60	12	9	0.9959	0.9964	0.9956	0.9961	0.9956	0.9961
		16	0.9963	0.9974	0.9959	0.9974	0.9959	0.9971
		20	0.9959	0.9961	0.9956	0.9958	0.9956	0.9958
		40	0.9959	0.9968	0.9959	0.9965	0.9959	0.9965

Table 13: Synthetic Type A/B (K=2), dimension d=3: total accuracy of the SW k-means clustering with UnifOrtho projections across window size h_1, step h_2 and number of projections $L \in \{9, 16, 20, 40\}$. Dark red $\approx$ 1, pale $\approx$ 0.5.

h_1	h_2	L	Max		Median		Max(MCCD)	
			A	B	A	B	A	B
20	4	9	0.9866	0.9986	0.9866	0.9986	0.9866	0.9986
		16	0.9844	0.9985	0.9843	0.9985	0.9843	0.9985
		20	0.9867	0.9985	0.9866	0.9984	0.9865	0.9985
		40	0.9872	0.9988	0.9872	0.9988	0.9872	0.9988
30	6	9	0.9967	0.9981	0.9966	0.9980	0.9965	0.9979
		16	0.9931	0.9983	0.9930	0.9982	0.9930	0.9982
		20	0.9942	0.9982	0.9942	0.9981	0.9940	0.9981
		40	0.9943	0.9982	0.9943	0.9982	0.9943	0.9981
40	8	9	0.9954	0.9979	0.9953	0.9979	0.9954	0.9979
		16	0.9965	0.9981	0.9964	0.9979	0.9963	0.9979
		20	0.9969	0.9977	0.9968	0.9975	0.9967	0.9975
		40	0.9967	0.9981	0.9965	0.9979	0.9965	0.9979
50	10	9	0.9955	0.9973	0.9952	0.9973	0.9955	0.9973
		16	0.9948	0.9973	0.9948	0.9972	0.9948	0.9973
		20	0.9950	0.9973	0.9947	0.9971	0.9947	0.9973
		40	0.9948	0.9973	0.9946	0.9972	0.9946	0.9973
60	12	9	0.9951	0.9970	0.9951	0.9967	0.9948	0.9967
		16	0.9949	0.9972	0.9949	0.9972	0.9946	0.9969
		20	0.9951	0.9967	0.9951	0.9966	0.9951	0.9964
		40	0.9951	0.9966	0.9949	0.9963	0.9948	0.9966

Table 14: Synthetic Type A/B (K=2), dimension d=4: total accuracy of the SW k-means clustering with UnifOrtho projections across window size h_1, step h_2 and number of projections $L \in \{9, 16, 20, 40\}$. Dark red $\approx$ 1, pale $\approx$ 0.5.

h_1	h_2	L	Max		Median		Max(MCCD)	
			C	D	C	D	C	D
20	4	2	0.5328	0.8590	0.5304	0.5054	0.5304	0.8590
		4	0.9763	0.8423	0.5211	0.5172	0.9758	0.8423
		9	0.5310	0.8652	0.5271	0.5104	0.5271	0.8627
		16	0.9789	0.8518	0.5303	0.5088	0.9789	0.8499
30	6	2	0.9943	0.8800	0.9941	0.5146	0.9940	0.8800
		4	0.9936	0.8770	0.5481	0.5161	0.9929	0.8758
		9	0.8878	0.8746	0.5468	0.5061	0.8793	0.8746
		16	0.9927	0.5159	0.5429	0.5134	0.9922	0.5159
40	8	2	0.9906	0.8818	0.5506	0.5238	0.9897	0.8816
		4	0.9950	0.8819	0.9946	0.5270	0.9946	0.8814
		9	0.8723	0.8774	0.5449	0.5184	0.8723	0.8774
		16	0.9946	0.8812	0.7702	0.5268	0.9946	0.8812
50	10	2	0.8769	0.5277	0.5731	0.5166	0.8761	0.5035
		4	0.5756	0.5330	0.5736	0.5287	0.5739	0.5277
		9	0.9951	0.8951	0.5786	0.5254	0.9948	0.8939
		16	0.9942	0.8920	0.5765	0.5219	0.9942	0.8908
60	12	2	0.8909	0.9950	0.5711	0.5148	0.8909	0.9946
		4	0.9947	0.9027	0.7822	0.5324	0.9944	0.9027
		9	0.9944	0.8918	0.5800	0.6979	0.9941	0.8901
		16	0.9964	0.9032	0.9958	0.5240	0.9955	0.9032

Table 15: Synthetic Type C/D (K=3), dimension d=2: total accuracy of the SW k-means clustering with UnifOrtho projections across window size h_1, step h_2 and number of projections $L \in \{2, 4, 9, 16\}$. Dark red ≈ 1, pale ≈ 0.5.

h_1	h_2	L	Max		Median		Max(MCCD)	
			C	D	C	D	C	D
20	4	9	0.9825	0.8584	0.5422	0.5120	0.9825	0.8579
		16	0.9852	0.5174	0.5404	0.5144	0.9852	0.5148
		20	0.9860	0.5202	0.9858	0.5170	0.9858	0.5186
		40	0.9849	0.8612	0.7620	0.5137	0.9849	0.8608
30	6	9	0.9930	0.8730	0.9928	0.5063	0.9930	0.8713
		16	0.9930	0.5150	0.5843	0.5121	0.9930	0.5097
		20	0.9931	0.8745	0.5758	0.5056	0.9931	0.8745
		40	0.9931	0.8763	0.5786	0.5075	0.9930	0.8752
40	8	9	0.9939	0.8832	0.5690	0.5032	0.9939	0.8832
		16	0.9943	0.8832	0.7807	0.5039	0.9943	0.8830
		20	0.9948	0.8781	0.9944	0.5057	0.9944	0.8781
		40	0.9941	0.8846	0.5653	0.5066	0.9939	0.8819
50	10	9	0.9954	0.8779	0.9949	0.5050	0.9954	0.8776
		16	0.5649	0.8798	0.5617	0.5072	0.5603	0.8782
		20	0.9957	0.8772	0.7788	0.5103	0.9952	0.8771
		40	0.9955	0.8783	0.5552	0.5092	0.9955	0.8783
60	12	9	0.9952	0.5169	0.5590	0.5066	0.9952	0.5061
		16	0.9959	0.8767	0.9955	0.5126	0.9959	0.8767
		20	0.9952	0.8789	0.9949	0.5156	0.9949	0.8789
		40	0.9955	0.8746	0.5660	0.5111	0.9952	0.8746

Table 16: Synthetic Type C/D (K=3), dimension d=3: total accuracy of the SW k-means clustering with UnifOrtho projections across window size h_1, step h_2 and number of projections $L \in \{9, 16, 20, 40\}$. Dark red $\approx$ 1, pale $\approx$ 0.5.

h_1	h_2	L	Max		Median		Max(MCCD)	
			C	D	C	D	C	D
20	4	9	0.9895	0.8652	0.5501	0.7030	0.9895	0.8644
		16	0.9873	0.8660	0.7748	0.5388	0.9872	0.8648
		20	0.9887	0.8682	0.7770	0.5446	0.9884	0.8682
		40	0.5707	0.5465	0.5677	0.5437	0.5669	0.5462
30	6	9	0.9935	0.8738	0.5788	0.5284	0.9935	0.8738
		16	0.9935	0.8735	0.5737	0.5258	0.9935	0.8735
		20	0.9942	0.8720	0.5778	0.5240	0.9942	0.8720
		40	0.9942	0.8718	0.7824	0.5265	0.9940	0.8718
40	8	9	0.9950	0.8732	0.5569	0.5241	0.9949	0.8732
		16	0.9952	0.8717	0.5637	0.5375	0.9952	0.8712
		20	0.9963	0.8723	0.5555	0.5399	0.9958	0.8723
		40	0.9955	0.8719	0.5649	0.5419	0.9952	0.8719
50	10	9	0.9946	0.8773	0.5689	0.7008	0.9946	0.8771
		16	0.9947	0.5239	0.5556	0.5223	0.9937	0.5211
		20	0.9950	0.8741	0.5553	0.5247	0.9950	0.8738
		40	0.9950	0.8732	0.5602	0.5276	0.9937	0.8724
60	12	9	0.9944	0.8733	0.5767	0.5405	0.9940	0.8733
		16	0.9936	0.8757	0.5622	0.5437	0.9929	0.8757
		20	0.9948	0.8764	0.5591	0.5447	0.9948	0.8746
		40	0.9955	0.8744	0.5688	0.5451	0.9955	0.8744

Table 17: Synthetic Type C/D (K=3), dimension d=4: total accuracy of the SW k-means clustering with UnifOrtho projections across window size h_1, step h_2 and number of projections $L \in \{9, 16, 20, 40\}$. Dark red $\approx$ 1, pale $\approx$ 0.5.

h_1	h_2	L	Max		Median		Max(MCCD)	
			A	B	A	B	A	B
20	4	9	0.9985	0.9988	0.9985	0.9987	0.9985	0.9987
		20	0.9954	0.9990	0.9953	0.9990	0.9953	0.9990
		40	0.9955	0.9990	0.9955	0.9990	0.9955	0.9990
		100	0.9964	0.9991	0.9963	0.9991	0.9963	0.9991
30	6	9	0.9964	0.9986	0.9963	0.9985	0.9964	0.9986
		20	0.9976	0.9989	0.9975	0.9989	0.9976	0.9989
		40	0.9960	0.9989	0.9960	0.9987	0.9960	0.9987
		100	0.9974	0.9990	0.9973	0.9988	0.9973	0.9988
40	8	9	0.9978	0.9977	0.9976	0.9975	0.9978	0.9977
		20	0.9973	0.9984	0.9973	0.9984	0.9973	0.9984
		40	0.9968	0.9984	0.9968	0.9984	0.9966	0.9984
		100	0.9973	0.9984	0.9973	0.9984	0.9973	0.9984
50	10	9	0.9964	0.9982	0.9964	0.9982	0.9964	0.9980
		20	0.9972	0.9980	0.9969	0.9977	0.9972	0.9977
		40	0.9960	0.9973	0.9959	0.9972	0.9958	0.9970
		100	0.9969	0.9978	0.9966	0.9978	0.9966	0.9976
60	12	9	0.9950	0.9978	0.9948	0.9977	0.9950	0.9978
		20	0.9970	0.9974	0.9968	0.9973	0.9967	0.9971
		40	0.9966	0.9976	0.9963	0.9973	0.9963	0.9973
		100	0.9969	0.9981	0.9969	0.9981	0.9969	0.9981

Table 18: Synthetic Type A/B (K=3), dimension d=100: total accuracy of the SW k-means clustering with UnifOrtho projections across window size h_1, step h_2 and number of projections $L \in \{9, 20, 40, 100\}$. Dark red ≈ 1, pale ≈ 0.5.

h_1	h_2	L	Max		Median		Max(MCCD)	
			C	D	C	D	C	D
20	4	9	0.9966	0.8734	0.5821	0.5558	0.9965	0.8728
		20	0.9954	0.8775	0.5722	0.5575	0.9949	0.8755
		40	0.9981	0.8754	0.5947	0.5510	0.9981	0.8751
		100	0.9960	0.8745	0.5909	0.5508	0.9959	0.8745
30	6	9	0.9972	0.8785	0.5749	0.5586	0.9972	0.8774
		20	0.9973	0.8803	0.6002	0.5578	0.9963	0.8797
		40	0.9976	0.8841	0.7981	0.5526	0.9966	0.8836
		100	0.9967	0.8825	0.6028	0.5536	0.9962	0.8780
40	8	9	0.9941	0.8747	0.6009	0.5538	0.9933	0.8741
		20	0.9963	0.8791	0.6280	0.5560	0.9963	0.8791
		40	0.9973	0.8827	0.6226	0.5577	0.9973	0.8780
		100	0.9968	0.8806	0.6208	0.5570	0.9968	0.8779
50	10	9	0.9939	0.8762	0.6139	0.5735	0.9939	0.8754
		20	0.9953	0.8836	0.6050	0.5666	0.9942	0.8836
		40	0.9965	0.8796	0.6094	0.5589	0.9965	0.8756
		100	0.9957	0.8832	0.6259	0.5624	0.9957	0.8761
60	12	9	0.9963	0.8775	0.9947	0.5695	0.7746	0.8758
		20	0.9947	0.8881	0.6101	0.5588	0.9939	0.8864
		40	0.9938	0.8823	0.6282	0.5538	0.9935	0.8821
		100	0.9950	0.8804	0.6151	0.5603	0.9942	0.8781

Table 19: Synthetic Type C/D (K=3), dimension d=100: total accuracy of the SW k-means clustering with UnifOrtho projections across window size h_1, step h_2 and number of projections $L \in \{9, 20, 40, 100\}$. Dark red ≈ 1, pale ≈ 0.5.

A.1.1 Look-ahead regime strategy

Algorithm 2: LOOKAHEADSTRATUNIFORTHO — Regime Strategy with Look-Ahead Bias

Input: Initial capital C_0,;
number of simulations N_S,;
price matrix $S \in \mathbb{R}^{T \times d}$,;
number of projections L,;
trimming parameters h_1, h_2,;
window size w,;
number of clusters K=2,;
risk metric m,;
weighting scheme
Output: Portfolio value path V,;
signal vector s,;
cumulative PnL

$\varepsilon \leftarrow 10^{-6}$;
if *weighting* = `"equal"` **then**
  $R \leftarrow \mathrm{PctReturns}(S)$; ▷ `drop first (NaN) row`
  $\theta \leftarrow \mathbf{1}_d / d$; ▷ `equal weights`
  $r^{(\mathrm{pf})} \leftarrow R\,\theta^\top$; ▷ `row-wise sum`
else
  $R \leftarrow \mathrm{PctReturns}(S)$;
  $\sigma \leftarrow \mathrm{RollingStd}(R, w)$, keep rows $\geq w-1$;
  $u \leftarrow \mathbf{1}/(\sigma + \varepsilon)$; ▷ `inverse vol, elementwise`
  $\theta_t \leftarrow u_t / \sum_j u_{t,j}$; ▷ `normalize each row to sum 1`
  $r_t^{(\mathrm{pf})} \leftarrow \sum_j R_{t,j}\,\theta_{t,j}$; ▷ `aligned from row` $w-1$
$\hat{F},\ \boldsymbol{\mu},\ \boldsymbol{\lambda} \leftarrow \mathrm{MAXMCCD\_UNIFORTHO}(N_S, S, K, L, \varepsilon, h_1, h_2, m)$;
for *each t* **do**
  if $\lambda_t = 1$ **then**
    $s_t \leftarrow +1$; ▷ `Bullish` $\Rightarrow$ `Long`
  else
    $s_t \leftarrow -1$; ▷ `Bearish` $\Rightarrow$ `Short`

▷ `Step 4 -- Apply signal & accounting`
$r_t^{(\mathrm{strat})} \leftarrow s_{w-h_1+t} \cdot r_t^{(\mathrm{pf})}$; ▷ `contemporaneous` $\rightarrow$ `look-ahead bias`
$\mathrm{cum}_t \leftarrow \prod_{k \leq t} \left(1 + r_k^{(\mathrm{strat})}\right)$;
$V_t \leftarrow C_0 \cdot \mathrm{cum}_t$;
$\mathrm{PnL}_t \leftarrow V_t - C_0$;
return V, s, PnL

A.1.2 Standard SW k-means trading strategy

Algorithm 3: LONG_STRAT_UNIFORTHO — Binary Regime Strategy

Input: Initial capital C_0,;
number of simulations N_S,;
price matrix $S \in \mathbb{R}^{T \times d}$,;
trimming parameters h_1, h_2,;
window size w,;
number of clusters K,;
risk metric m,;
majority look-back ℓ
Output: Portfolio value path V

$R \leftarrow \mathrm{PctReturns}(S)$; ▷ $R_t = S_t / S_{t-1} - 1$
$\theta \leftarrow \mathbf{1}_d / \sigma_d$; ▷ `inverse vol weighted returns`
$r^{(\mathrm{pf})} \leftarrow R\theta$; ▷ `portfolio return series`
$V \leftarrow [C_0]$;
$n \leftarrow \lfloor T/w \rfloor$;
for $i = 0$ **to** $n-2$ **do**
  $s \leftarrow i \cdot w, \quad e \leftarrow (i+1) \cdot w$;
  $W \leftarrow S[s:e]$; ▷ `current analysis window`
  $\hat{F},\ \boldsymbol{\mu},\ \boldsymbol{\lambda} \leftarrow$ MAXMCCD_UNIFORTHO$(N_S, W, K, L, \varepsilon, h_1, h_2, m)$;
  if $\ell > |\boldsymbol{\lambda}|$ **then**
    $\tilde{\boldsymbol{\lambda}} \leftarrow \boldsymbol{\lambda}$;
  else
    $\tilde{\boldsymbol{\lambda}} \leftarrow \boldsymbol{\lambda}_{[-\ell:]}$;
  for $k = 0, 1, \ldots, |\tilde{\boldsymbol{\lambda}}| - 1$ **do**
    $w_k \leftarrow \exp\left(-\frac{\ln 2}{\tau}\,(|\tilde{\boldsymbol{\lambda}}| - 1 - k)\right)$;
  $\hat{k} \leftarrow \arg\max_k \sum_{i=0}^{|\tilde{\boldsymbol{\lambda}}|-1} w_i \, \mathbf{1}\left[\tilde{\lambda}_i = k\right]$;
  if $\hat{k} = 1$ **then**
    $\sigma \leftarrow +1$; ▷ `Bullish ⇒ Long`
  else
    $\sigma \leftarrow -1$; ▷ `Bearish ⇒ Short`
  for *each* $r_t \in r^{(\mathrm{pf})}[e : e+w]$ **do**
    $V.\mathrm{append}\Big(V[-1] \times \big(1 + \sigma\, r_t\big)\Big)$;
return V

A.1.3 Implied Probability SW k-means trading strategy

Algorithm 4: LONG_STRAT_IMPLIED — Implied-Probability Regime Strategy

Input: Initial capital C_0; simulations N_S; price matrix $S \in \mathbb{R}^{T \times d}$; trimming h_1, h_2; window w; clusters K; metric m; signal type $\tau \in \{\texttt{continuous}, \texttt{hysteresis}, \texttt{conviction}\}$; entry/hold thresholds α_e, α_h; look-back ℓ; gradient flag/weight (g, ω_g)

Output: Portfolio value path V; signal history Σ

$R \leftarrow \mathrm{PctReturns}(S)$; $\theta \leftarrow \mathbf{1}_d \cdot 1/\sigma_d$; $r^{(\mathrm{pf})} \leftarrow R\,\theta$;
$V \leftarrow [C_0]$; $\Sigma \leftarrow []$; $n \leftarrow \lfloor T/w \rfloor$;
for $i = 0$ **to** $n-2$ **do**
  $s \leftarrow i \cdot w$; $e \leftarrow (i+1) \cdot w$; $W \leftarrow S[s:e]$;
  $\hat{F}, \boldsymbol{\mu}, \boldsymbol{\lambda} \leftarrow \textsc{MaxMCCD\_UnifOrtho}(N_S, W, K, L, \varepsilon, h_1, h_2, m)$;
  $\mathbf{P}, p_{\mathrm{switch}}, \mathbf{T}, \boldsymbol{\pi} \leftarrow \textsc{ComputeImpliedProba}(\hat{F}, \boldsymbol{\mu}, \boldsymbol{\lambda}, \ell, \mathrm{g}, \omega_g)$;
  $\hat{k} \leftarrow \arg\max_k \mathrm{count}(\boldsymbol{\lambda}_{[-h_2:]} = k)$;
  if $\tau = \mathit{continuous}$ **then**
    $\sigma \leftarrow \pi_{\mathrm{bull}} - \pi_{\mathrm{bear}}$;
    if $|\sigma| < \delta$ **then** $\sigma \leftarrow 0$ ($\triangleright$ dead zone);
  else if $\tau = \mathit{hysteresis}$ **then**
    $\sigma_{\mathrm{prev}} \leftarrow \Sigma[-1]$ **or** 0;
    if $\hat{k} = \mathit{Bull}$ **then**
      if $p_{switch} \geq \alpha_e$ ***and*** $\sigma_{prev} \geq 0$ **then** $\sigma \leftarrow -1$;
      else if $p_{switch} < \alpha_h$ ***and*** $\sigma_{prev} < 0$ **then** $\sigma \leftarrow +1$;
      else $\sigma \leftarrow 0$;
    else $\triangleright$ $\hat{k} = \texttt{Bear}$
      if $p_{switch} \geq \alpha_e$ ***and*** $\sigma_{prev} \leq 0$ **then** $\sigma \leftarrow +1$;
      else if $p_{switch} < \alpha_h$ ***and*** $\sigma_{prev} > 0$ **then** $\sigma \leftarrow -1$;
      else $\sigma \leftarrow 0$;
  else if $\tau = \mathit{conviction}$ **then**
    $d_k \leftarrow \begin{cases} +1 & \hat{k} = \mathrm{Bull} \\ -1 & \hat{k} = \mathrm{Bear} \end{cases}$;
    $c \leftarrow 1 - 1.5\, p_{\mathrm{switch}}$; $\triangleright$ `conviction decays with switch risk`
    $\sigma \leftarrow d_k \cdot c$;
  $\Sigma.\mathrm{append}(\sigma)$;
  for *each* $r_t \in r^{(\mathrm{pf})}[e : e+w]$ **do**
    $V.\mathrm{append}\big(V[-1] \times (1 + \sigma\, r_t)\big)$;
return V, Σ

The Mathematical Structure of Hadamard-Rademacher Matrices

Definition (from the paper [5]) The Hadamard-Rademacher chain on $O(2^L)$ is the Markov chain $(X_T)_{T=1}^{\infty}$ defined by:

$$X_T = \prod_{t=1}^{T} HD_t \tag{46}$$

where $(D_t)_{t=1}^{\infty}$ are independent random diagonal matrices with each diagonal element a Rademacher (i.e. $\mathrm{Unif}(\{+1,-1\})$) random variable, and H is the normalized Hadamard matrix, defined as the

Kronecker product:

$$H = \underbrace{\begin{pmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} \end{pmatrix} \otimes \cdots \otimes \begin{pmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} \end{pmatrix}}_{L \text{ times}} \tag{47}$$

These matrices X_T (typically with $T \in \{1,2,3\}$) have been used in dimensionality reduction, kernel approximation, and locality-sensitive hashing.

In practice one uses $T = 3$, giving the matrix:

$$M = HD_3 \cdot HD_2 \cdot HD_1 \tag{48}$$

where each $D_i = \mathrm{diag}(\varepsilon_1^{(i)}, \ldots, \varepsilon_d^{(i)})$ with $\varepsilon_j^{(i)} \overset{\text{i.i.d.}}{\sim} \mathrm{Unif}(\{+1,-1\})$.

References

[1] Donald W. K. Andrews. Heteroskedasticity and autocorrelation consistent covariance matrix estimation. *Econometrica*, 59(3):817–858, 1991.

[2] A. Ang and A. Timmermann. Regime changes and financial markets. *Annual Review of Financial Economics*, 4:313–337, 2012.

[3] J. Bai and P. Perron. Computation and analysis of multiple structural change models. *Journal of Applied Econometrics*, 97(1-2), 2003.

[4] Patric Bonnier, Patrick Kidger, Imanol Perez Arribas, Cristopher Salvi, and Terry Lyons. Deep signature transforms. *arXiv preprint arXiv:1905.08494*, 2019.

[5] K. Choromanski, M. Rowland, W. Chen, and A. Weller. *Unifying Orthogonal Monte Carlo Methods*, volume 97 of *Proceedings of Machine Learning Research*. PMLR, 2019.

[6] James D. Hamilton. A new approach to the economic analysis of nonstationary time series and the business cycle. *Econometrica*, 57(2):357–384, 1989.

[7] D. F. Hendry and J. A. Doornik. Empirical model discovery and theory evaluation. *Journal of Econometrics*, 178(2), 2014.

[8] B. Horvath, A. Muguruza, and M. Tomas. Wasserstein k-means for clustering distributions. *Journal of Machine Learning Research*, 2024.

[9] O. Ledoit and M. Wolf. Robust performance hypothesis testing with the sharpe ratio. *Journal of Empirical Finance*, 15(5):850–859, 2008.

[10] Qinmeng Luan and James Hamp. Automated regime classification in multidimensional time series data using sliced wasserstein k-means clustering. *Data Science in Finance and Economics*, 5(3):387–418, 2025.

[11] Scott M Lundberg and Su-In Lee. A unified approach to interpreting model predictions. 2017.

[12] Christoph Memmel. Performance hypothesis testing with the sharpe ratio. *Finance Letters*, 1(1):29–40, 2003.

[13] Francesco Mezzadri. How to generate random matrices from the classical compact groups. *Notices of the American Mathematical Society*, 54(5):592–604, 2007. arXiv:math-ph/0609050.

[14] Mark Rowland, Jiri Hron, Yunhao Tang, Krzysztof Choromanski, Tamas Sarlos, and Adrian Weller. Orthogonal estimation of Wasserstein distances. In *Proceedings of the 22nd International Conference on Artificial Intelligence and Statistics (AISTATS)*, volume 89 of *Proceedings of Machine Learning Research*, pages 186–195. PMLR, 2019.

[15] T. T. Rydén, Teräsvirta and S. Åsbrink. Stylized facts of daily return series and the hidden markov model. *Computational Statistics & Data Analysis*, 28(3):217–236, 1998.

[16] Rémi Bardenet Vladimir Petrovic and Agnès Desolneux. Repulsive monte carlo on the sphere for the sliced wasserstein distance. *arXiv preprint arXiv:2509.10166*, 2025.